
O-RAN Work Group 10 (OAM for O-RAN)

Topology Exposure and Inventory Application Protocols Specification - Stage 3

Copyright © 2026 by the O-RAN ALLIANCE e.V.

The copying or incorporation into any other work of part or all of the material available in this specification in any form without the prior written permission of O-RAN ALLIANCE e.V. is prohibited, save that you may print or download extracts of the material of this specification for your personal use, or copy the material of this specification for the purpose of sending to individual third parties for their information provided that you acknowledge O-RAN ALLIANCE as the source of the material and that you inform the third party that these conditions apply to them and that they must comply with them.

O-RAN ALLIANCE e.V., Buschkauler Weg 27, 53347 Alfter, Germany

Register of Associations, Bonn VR 11238, VAT ID DE321720189

Contents

Foreword.....	3
Modal verbs terminology.....	3
Executive summary	3
1. Scope	4
2. References	4
2.1. Normative references	4
2.2. Informative references.....	4
3. Definition of terms, symbols and abbreviations.....	5
3.1. Terms.....	5
3.2. Symbols.....	5
3.3. Abbreviations	5
4 Application protocol for TE&IV services	5
4.1 Introduction	5
4.2 Version conventions for the present document	5
5 RESTful TE&IV Service APIs.....	5
5.1 Overview	5
5.2 Versioning of RESTful TE&IV Service APIs.....	6
5.3 URI structure and supported content formats.....	6
5.4 General considerations for RESTful TE&IV Service APIs.....	7
5.4.1 Usage of HTTP header fields.....	7
5.4.2 Handling of large query results.....	7
5.4.3 Error reporting	7
6 Topology Exposure and Inventory Management Services.....	7
6.1 Topology Inventory API	7
6.1.1 Introduction.....	7
6.1.2 API version	7
6.1.3 Resource structure and methods	7
6.1.4 Service operations.....	9
6.1.5 Resources.....	19
6.1.6 Custom operations without associated resources	37
6.1.7 Notifications	38
6.1.8 Data model.....	39
6.1.9 Error Handling.....	43
Annex A (normative): OpenAPI Specifications	43
A.1 Overview	43
A.2 Topology & Inventory API	43
Annex (informative): Change History	60

Foreword

This Technical Specification (TS) has been produced by WG10 of the O-RAN ALLIANCE.

The content of the present document is subject to continuing work within O-RAN and may change following formal O-RAN approval. Should the O-RAN ALLIANCE modify the contents of the present document, it will be re-released by O-RAN with an identifying change of version date and an increase in version number as follows:

version xx.yy.zz

where:

- xx: the first digit-group is incremented for all changes of substance, i.e. technical enhancements, corrections, updates, etc. (the initial approved document will have xx=01). Always 2 digits with leading zero if needed.
- yy: the second digit-group is incremented when editorial only changes have been incorporated in the document. Always 2 digits with leading zero if needed.
- zz: the third digit-group included only in working versions of the document indicating incremental changes during the editing process. External versions never include the third digit-group. Always 2 digits with leading zero if needed.

Modal verbs terminology

In the present document "**shall**", "**shall not**", "**should**", "**should not**", "**may**", "**need not**", "**will**", "**will not**", "**can**" and "**cannot**" are to be interpreted as described in clause 3.2 of the O-RAN Drafting Rules (Verbal forms for the expression of provisions).

"**must**" and "**must not**" are **NOT** allowed in O-RAN deliverables except when used in direct citation.

Executive summary

The present document specifies the TE&IV APIs suited to realize the use cases of TE&IV Service Consumer as specified in TE&IV UCR [1].

1. Scope

This document specifies the TE&IV APIs used to support TE&IV services within the SMO.

2. References

2.1. Normative references

References are either specific (identified by date of publication and/or edition number or version number) or non-specific. For specific references, only the cited version applies. For non-specific references, the latest version of the referenced document (including any amendments) applies. In the case of a reference to a 3GPP document, a non-specific reference implicitly refers to the latest version of that document in Release 18.

NOTE: While any hyperlinks included in this clause were valid at the time of publication, O-RAN cannot guarantee their long term validity.

The following referenced documents are necessary for the application of the present document.

- [1] O-RAN.WG10.TE&IV-UCR.0: "Topology Exposure and Inventory Management Services Use Cases and Requirement Specification" ("TE&IV UCR")
- [2] 3GPP TS 29.500: "5G System; Technical Realization of Service Based Architecture; Stage 3"
- [3] 3GPP TS 29.501: "5G System; Principles and Guidelines for Services Definition"
- [4] Semantic Versioning 2.0.0: <https://semver.org>
- [5] IETF RFC 8259: "The JavaScript Object Notation (JSON) Data Interchange Format"
- [6] IETF RFC 3986: "Uniform Resource Identifier (URI): Generic Syntax"
- [7] ETSI GS NFV-SOL 013 Rel3: "Protocols and Data Models; Specification of common aspects for RESTful NFV MANO APIs"
- [8] IETF RFC 7951: "JSON Encoding of Data Modeled with YANG"
- [9] O-RAN.WG10.TS.TE&IV-CIMI.0: "Topology Exposure & Inventory Common Information Models and Interface Specification - Stage 2" ("TE&IV CIMI")
- [10] O-RAN.WG10.TS.TE&IV-DM.0: "Topology Exposure and Inventory Data Model Specification - Stage 3" ("TE&IV DM")
- [11] OpenAPI: "OpenAPI 3.0.3 Specification", <http://spec.openapis.org/oas/v3.0.3.html>

2.2. Informative references

References are either specific (identified by date of publication and/or edition number or version number) or non specific. For specific references, only the cited version applies. For non-specific references, the latest version of the referenced document (including any amendments) applies. In the case of a reference to a 3GPP document, a non-specific reference implicitly refers to the latest version of that document in Release 18, or the latest 3GPP release prior to Release 18 that includes that document.

NOTE: While any hyperlinks included in this clause were valid at the time of publication, O-RAN cannot guarantee their long-term validity.

The following referenced documents are not necessary for the application of the present document, but they assist the user with regard to a particular subject area.

- [i.1] IETF RFC 4229: "HTTP Header Field Registrations"

3. Definition of terms, symbols and abbreviations

3.1. Terms

For the purpose of the present document, the following terms apply:

API Consumer: The TE&IV Service Consumer consuming one or more services using APIs.

API Producer: The TE&IV Service Producer that offers its services for consumption via APIs.

Domain: A logical grouping of topology entities and/or topology relationships.

3.2. Symbols

Void

3.3. Abbreviations

Void

4 Application protocol for TE&IV services

4.1 Introduction

The present document contains a realization of the services identified in TE&IV UCR [1]. This definition of the TE&IV Application Protocols Interface (TE&IV API) defined in the present document is based on the 3GPP service framework for network functions specified in 3GPP TS 29.501 [3].

4.2 Version conventions for the present document

The version number of the present document follows the "xx.yy" versioning scheme. There could be implications for the interoperability between TE&IV Service Consumers and TE&IV Service API implementations in the SMO that are based on different versions of the present document.

An incremented "xx" version field of the present document could indicate that a new major feature (e.g., a new TE&IV Service) has been added or that an incompatible change has been made to one or more TE&IV Service APIs. An incremented "yy" version field could indicate that an optional feature has been added, a technical issue has been fixed, or that clarifications or editorial corrections have been made.

The version conventions for RESTful TE&IV Service APIs are defined in clause 5.2.

5 RESTful TE&IV Service APIs

5.1 Overview

The design of the RESTful TE&IV Service APIs is based on the services and requirements defined in TE&IV UCR [1], and on the protocol design framework as specified in 3GPP TS 29.501 [3].

The present document defines the protocols for the TE&IV service APIs listed in table 5.1-1.

Table 5.1-1: RESTful TE&IV Service APIs and their versions defined in the present document.

Service API	API version
Topology & Inventory API	1.0.0

5.2 Versioning of RESTful TE&IV Service APIs

Each RESTful TE&IV Service API is versioned independently. The API version number defined in the present document contains three numerical fields following a MAJOR.MINOR.PATCH pattern, and may contain a pre-release version field, according to SemVer [4].

The API version number held by an implementation may additionally include a build metadata field, according to SemVer [4], to indicate a specific deployment. The content of this field is implementation specific; it is provided by the deployment. The <apiVersion> path segment used in URI structures indicate the MAJOR field of the API version number. The full API version number is visible in the "version" field of the "info" object of each OpenAPI document in Annex A.

To indicate the full API version the API Consumer intends to use, the API Consumer may include the "Version" HTTP header (see IETF RFC 4229 [i.1]) in an HTTP request, in which case the header shall contain the version identifier as defined above. It is optional to include the build metadata field.

The API Producer shall include in the response the "Version" HTTP header signalling the used API version, including the build metadata if available. If the build metadata have been omitted in the request, the API Producer shall use the combination of MAJOR, MINOR, PATCH, and pre-release indicator as requested and the highest supported value for the build metadata field for that combination, if available. In case the API Consumer has not sent a "Version" header in the request, the API Producer shall use the latest available version, and signal it in the "Version" header.

NOTE: In case multiple versions are supported by an API Producer under the URI for a major version, this allows the API Consumer to request a particular version. This mechanism is referred to as "microversioning".

If the API version signalled by the API Consumer in the "Version" request header is not supported by the API Producer, the API Producer shall respond with a "406 Not Acceptable" error and may include in the response payload body a Problem Details structure providing more information on the cause of the error.

5.3 URI structure and supported content formats

This clause specifies the URI prefix and the supported content formats applicable to the RESTful TE&IV Service APIs.

All resource URIs of the APIs shall have the following prefix:

{apiRoot}/<apiName>/<apiVersion>/

The request URIs used in HTTP requests from the API Consumer towards the API Producer shall have the resource URI structure defined in clause 4.4.1 of 3GPP TS 29.501 [3], i.e.:

{apiRoot}/<apiName>/<apiVersion>/<apiSpecificResourceUriPart>

with the following components:

- The {apiRoot} shall be set as described in clause 4.4.1 of 3GPP TS 29.501 [3]; however, the restrictions w.r.t the operator specific FQDN of the host portion defined there do not apply.
- The <apiName> indicates the API name of the service interface in an abbreviated form. It is defined in the clause specifying the corresponding RESTful TE&IV Service API.
- The <apiVersion> indicates the major version (see clause 5.2) of the API and is defined in the clause specifying the corresponding RESTful TE&IV ServiceAPI.
- Each <apiSpecificResourceUriPart> represents a specific resource of the API. It is defined in the corresponding RESTful TE&IV Service API for each one of the defined resources.

For HTTP requests and responses that have message content, the content format JSON shall be supported (see IETF RFC 8259 [5]). The JSON content shall be signalled by the content type "application/json" and for JSON encoded YANG, the content type "application/yang.data+json" shall be used (see IETF RFC 7951 [8]).

All resource URIs of the API shall comply with the URI syntax as defined in IETF RFC 3986 [6]. An implementation that dynamically generates resource URI parts (individual path segments, sequences of path segments that are separated by "/", query parameter values) shall ensure that these parts only use the character set that is allowed by IETF RFC 3986 [6] for these parts.

5.4 General considerations for RESTful TE&IV Service APIs

5.4.1 Usage of HTTP header fields

HTTP headers are components of the headers section of the HTTP request and response messages. The usage of HTTP header fields shall follow the definitions in clause 4.2 of ETSI GS NFV-SOL 013 [7].

5.4.2 Handling of large query results

The handling of large query results shall be supported by RESTful TE&IV Service APIs as specified in clause 5.4.2 of ETSI GS NFV-SOL 013 [7].

5.4.3 Error reporting

In RESTful interfaces, application errors are mapped to HTTP errors. Since HTTP error information is generally not enough to discover the root cause of the error, additional application specific error information is typically delivered in the message content based on the ProblemDetails data type.

HTTP error responses shall be supported as specified in clause 4.8 of 3GPP TS 29.501 [3]. Protocol errors and application errors specified in table 5.2.7.1-1 of 3GPP TS 29.500 [2] shall be supported for an HTTP method if the corresponding HTTP status codes are specified as mandatory for that HTTP method in table 5.2.7.1-1 of 3GPP TS 29.500 [2].

6 Topology Exposure and Inventory Management Services

6.1 Topology Inventory API

6.1.1 Introduction

This API enables the API Consumer to query the topology entities, entity types and its relationship to other topology entities as defined in TE&IV CIMI [9].

6.1.2 API version

For the Topology Inventory API as specified in the present document, the MAJOR version field shall be 1, the MINOR version field shall be 0 and the PATCH version field shall be 0 (see clause 4.3.1.1 of 3GPP TS 29.501 [3] for a definition of the version fields). Consequently, the <apiVersion> URI path segment shall be set to "v1".

6.1.3 Resource structure and methods

The request URIs used in HTTP requests from the API Consumer towards the API Producer shall have the resource URI structure as defined in clause 5.2. The <apiName> resource URI variable shall be "topology-inventory". The <apiSpecificResourceUriPart> for each resource shall be set as described in clause 6.1.5.

Figure 6.1.3-1 shows the overall resource URI structure defined for the Topology Inventory API.

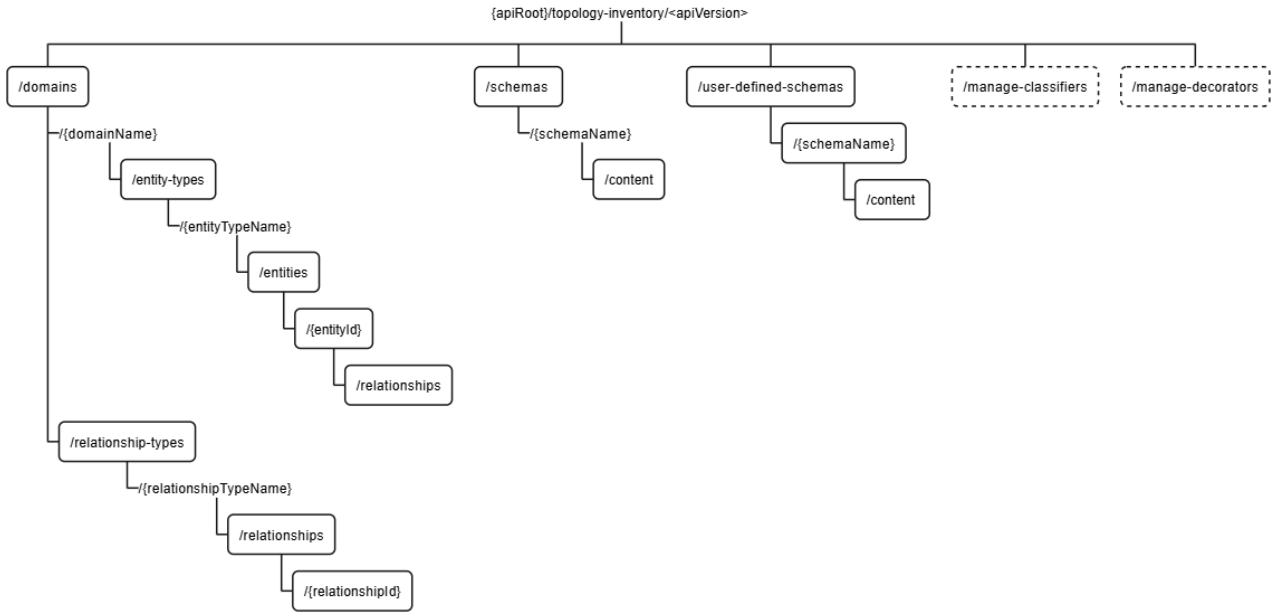


Figure 6.1.3-1: Resource URI structure of the Topology Inventory API

Table 6.1.3-1 lists the individual resources defined for the API, the applicable HTTP methods, and the associated service operations.

Table 6.1.3-1: Resources and methods overview of the Topology Inventory API

Resource name	Resource URI	HTTP method	Service Operation
All topology domains	.../domains	GET	getAllDomains
All topology entity types in a topology domain	.../domains/{domainName}/entity-types	GET	getTopologyEntityTypes
All instances of a topology entity type in a topology domain	.../domains/{domainName}/entity-types/{entityTypeName}/entities	GET	getTopologyByEntityTypeName
Individual instance of a topology entity type in a topology domain	.../domains/{domainName}/entity-types/{entityTypeName}/entities/{entityId}	GET	getTopologyById
All relationships of an instance of a topology entity type in a topology domain	.../domains/{domainName}/entity-types/{entityTypeName}/entities/{entityId}/relationships	GET	getAllRelationshipsForEntityId
All topology relationship types in a topology domain	.../domains/{domainName}/relationship-types	GET	getTopologyRelationshipTypes
Individual topology relationship	.../domains/{domainName}/relationship-types/{relationshipTypeName}/relationships	GET	getRelationshipsByType
Individual topology relationship instance	.../domains/{domainName}/relationship-types/{relationshipTypeName}/relationships/{relationshipId}	GET	getRelationshipById
All topology model schemas	.../schemas	GET	getSchemas
Individual model schema content	.../schemas/{schemaName}/content	GET	getSchemaByName
All user defined schemas	.../user-defined-schemas	GET POST	getUserDefinedSchemas createUserDefinedSchema
User defined schema instance	.../user-defined-schemas/{schemaName}	DELETE	deleteUserDefinedSchema
Content of user defined schema instance	.../user-defined-schemas/{schemaName}/content	GET	getUserDefinedSchemaByName

Table 6.1.3-2: Custom operations without associated resources

Custom operation URI	Mapped HTTP method	Description
{apiRoot}/topology-inventory/<apiVersion>/manage-classifiers	POST	Update or delete specified entities and/or relationships with classifiers.
{apiRoot}/topology-inventory/<apiVersion>/manage-decorators	POST	Update or delete specified entities and/or relationships with decorators.

6.1.4 Service operations

6.1.4.1 Operation getAllDomains

The API Consumer uses this operation to discover the available topology domains

```
@startuml
autonumber
Participant "API Consumer" as Consumer
```

```

Participant "API Producer" as Producer
Consumer ->> Producer: GET .../domains
Producer -->> Consumer: 200 OK (Domains)
@enduml

```

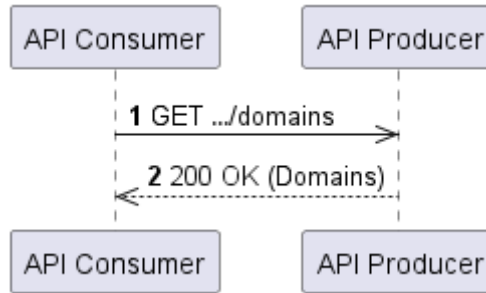


Figure 6.1.4.1-1: getDomains Operation

The service operation is as follows:

1. The API Consumer shall send an HTTP GET request to the API Producer. The target URI shall identify the resource (.../domains) to discover the available topology domains.
2. The API Producer shall return the HTTP GET response. On success, "200 OK" shall be returned and the message content shall carry a topology domain information. On failure, the appropriate error code shall be returned, and the response message content may contain additional error information.

6.1.4.2 Operation getTopologyEntityTypes

The API Consumer uses this operation to discover the topology entity types in a topology domain.

```

@startuml
autonumber
Participant "API Consumer" as Consumer
Participant "API Producer" as Producer
Consumer ->> Producer: GET .../domains/{domainName}/entity-types
Producer -->> Consumer: 200 OK (EntityTypes)
@enduml

```

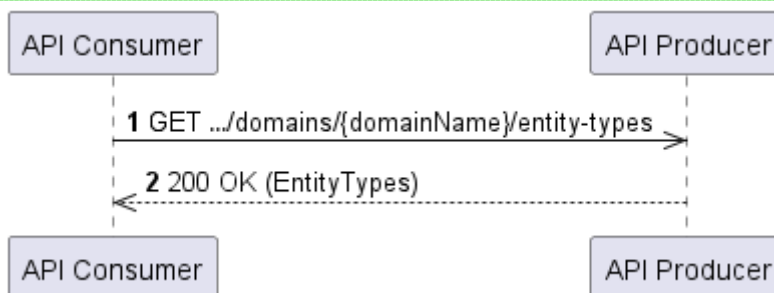


Figure 6.1.4.2-1: getTopologyEntityTypes Operation

The service operation is as follows:

1. The API Consumer shall send an HTTP GET request to the API Producer. The target URI shall identify the resource (.../domains/{domainName}/entity-types) to discover the topology entities within a topology domain.
2. The API Producer shall return the HTTP GET response. On success, "200 OK" shall be returned and the message content shall carry a topology entity types information within a topology domain. On failure, the appropriate error code shall be returned, and the response message content may contain additional error information.

6.1.4.3 Operation getTopologyByEntityTypeName

The API Consumer uses this operation to query all instances of a topology entity type in a topology domain

```
@startuml
autonumber
Participant "API Consumer" as Consumer
Participant "API Producer" as Producer
Consumer ->> Producer: GET .../domains/{domainName}/entity-types/{entityTypeName}/entities
Producer -->> Consumer: 200 OK (Entities)
@enduml
```



Figure 6.1.4.3-1: getTopologyByEntityTypeName Operation

The service operation is as follows:

1. The API Consumer shall send an HTTP GET request to the API Producer. The target URI shall identify the resource (.../domains/{domainName}/entity-types/{entityTypeName}/entities) to query the instances of a specific topology entity type within a topology domain.
2. The API Producer shall return the HTTP GET response. On success, "200 OK" shall be returned and the message content shall carry the instances of a specific topology entity type information within a topology domain. On failure, the appropriate error code shall be returned, and the response message content may contain additional error information.

6.1.4.4 Operation getTopologyById

The API Consumer uses this operation to query an instance of a topology entity type in a topology domain

```
@startuml
autonumber
Participant "API Consumer" as Consumer
Participant "API Producer" as Producer
Consumer ->> Producer: GET .../domains/{domainName}/entity-types/{entityTypeName}/entities/{entityId}
Producer -->> Consumer: 200 OK (EntityInstance)
@enduml
```

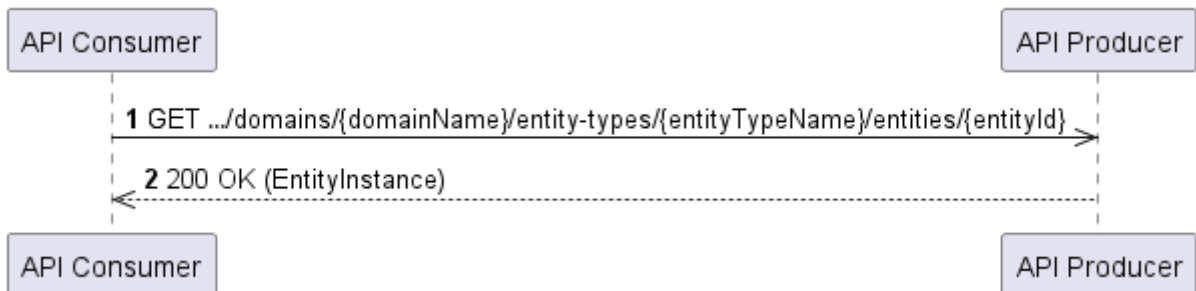


Figure 6.1.4.4-1: getTopologyById Operation

The service operation is as follows:

1. The API Consumer shall send an HTTP GET request to the API Producer. The target URI shall identify the resource (.../domains/{domainName}/entity-types/{entityTypeName}/entities/{entityId}) to query an instance of a specific topology entity type within a topology domain.
2. The API Producer shall return the HTTP GET response. On success, “200 OK” shall be returned and the message content shall carry an instance of a specific topology entity type information within a topology domain. On failure, the appropriate error code shall be returned, and the response message content may contain additional error information.

6.1.4.5 Operation getAllRelationshipsForEntityId

The API Consumer uses this operation to query the relationships of an instance of a topology entity type in a topology domain

```
@startuml
autonumber
Participant "API Consumer" as Consumer
Participant "API Producer" as Producer
Consumer ->> Producer: GET .../domains/{domainName}/entity-
types/{entityTypeName}/entities/{entityId}/relationships
Producer -->> Consumer: 200 OK (Relationships)
@enduml
```

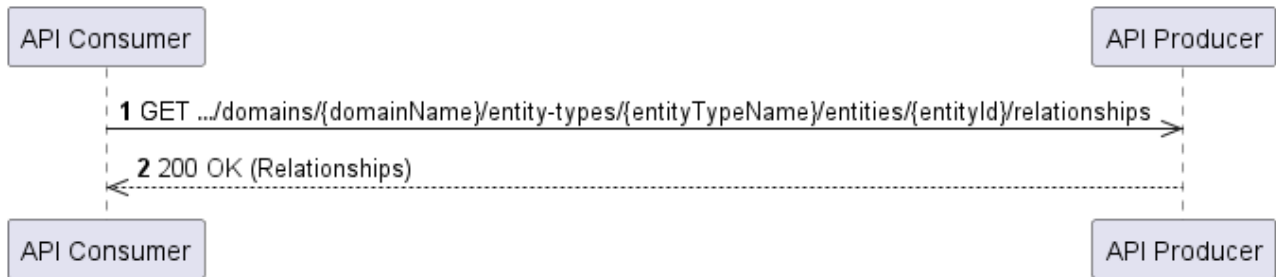


Figure 6.1.4.5-1: getAllRelationshipsForEntityId Operation

The service operation is as follows:

1. The API Consumer shall send an HTTP GET request to the API Producer. The target URI shall identify the resource (.../domains/{domainName}/entity-types/{entityTypeName}/entities/{entityId}/relationships) to query the relationships of an instance of a topology entity type within a topology domain.
2. The API Producer shall return the HTTP GET response. On success, “200 OK” shall be returned and the message content shall carry the relationships information of an instance of a topology entity type within in a topology domain. On failure, the appropriate error code shall be returned, and the response message content may contain additional error information.

6.1.4.6 Operation getTopologyRelationshipTypes

The API Consumer uses this operation to query all topology relationship types in a topology domain.

```
@startuml
autonumber
Participant "API Consumer" as Consumer
Participant "API Producer" as Producer
Consumer ->> Producer: GET .../domains/{domainName}/relationship-types
Producer -->> Consumer: 200 OK (RelationshipTypes)
@enduml
```

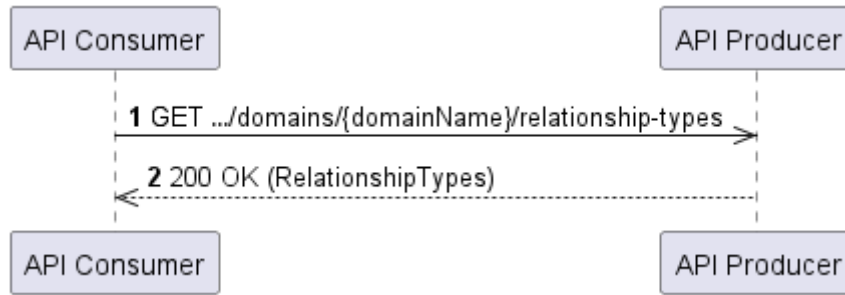


Figure 6.1.4.6-1: getTopologyRelationshipTypes operation

The service operation is as follows:

1. The API Consumer shall send an HTTP GET request to the API Producer. The target URI shall identify the resource (.../domains/{domainName}/relationship-types) to query all the topology relationship types of a topology entity within a topology domain.
2. The API Producer shall return the HTTP GET response. On success, “200 OK” shall be returned and the message content shall carry the topology relationship types information. On failure, the appropriate error code shall be returned, and the response message content may contain additional error information.

6.1.4.7 Operation getRelationshipsByType

The API Consumer uses this operation to query all topology relationship instances of a topology relationship type in a topology domain.

```

@startuml
autonumber
Participant "API Consumer" as Consumer
Participant "API Producer" as Producer
Consumer ->> Producer: GET .../domains/{domainName}/relationship-
types/{relationshipTypeName}/relationships
Producer -->> Consumer: 200 OK (Relationships)
@enduml
  
```

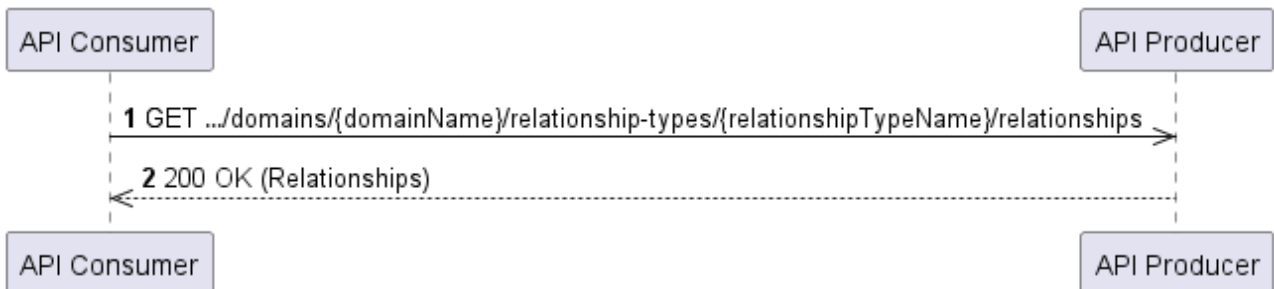


Figure 6.1.4.7-1: getRelationshipsByType operation

The service operation is as follows:

1. The API Consumer shall send an HTTP GET request to the API Producer. The target URI shall identify the resource (.../domains/{domainName}/relationship-types/{relationshipTypeName}/relationships) to query all topology relationship instances of a topology relationship type.
2. The API Producer shall return the HTTP GET response. On success, “200 OK” shall be returned and the message content shall carry an array of relationship information. On failure, the appropriate error code shall be returned, and the response message content may contain additional error information.

6.1.4.8 Operation getRelationshipById

The API Consumer uses this operation to query topology relationship instance of a topology relationship type in a topology domain.

```
@startuml
autonumber
Participant "API Consumer" as Consumer
Participant "API Producer" as Producer
Consumer ->> Producer: GET .../domains/{domainName}/relationship-
types/{relationshipTypeName}/relationships/{relationshipId}
Producer -->> Consumer: 200 OK (Relationship)
@enduml
```

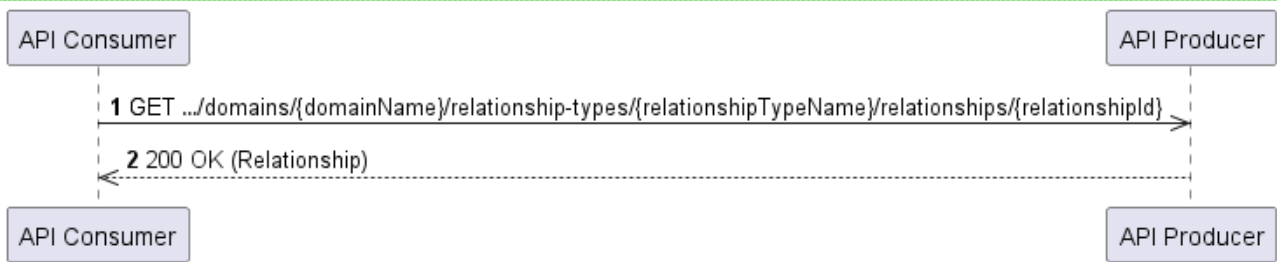


Figure 6.1.4.8-1: getRelationshipById operation

The service operation is as follows:

1. The API Consumer shall send an HTTP GET request to the API Producer. The target URI shall identify the resource (.../domains/{domainName}/relationship-types/{relationshipTypeName}/relationships/{relationshipId}) to query topology relationship instance of a topology relationship type.
2. The API Producer shall return the HTTP GET response. On success, "200 OK" shall be returned and the message content shall carry relationship information of a topology relationship type. On failure, the appropriate error code shall be returned, and the response message content may contain additional error information.

6.1.4.9 Operation getSchemas

The API Consumer uses this operation to discover the available schemas.

```
@startuml
autonumber
Participant "API Consumer" as Consumer
Participant "API Producer" as Producer
Consumer ->> Producer: GET .../schemas
Producer -->> Consumer: 200 OK (Schemas)
@enduml
```

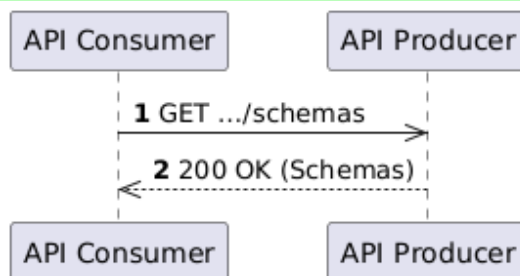


Figure 6.1.4.9-2: getSchemas Operation

The service operation is as follows:

1. The API Consumer shall send an HTTP GET request to the API Producer. The target URI shall identify the resource (.../schemas) to discover the available topology model schemas.
2. The API Producer shall return the HTTP GET response. On success, "200 OK" shall be returned and the message content shall carry a list of the available topology model schemas. On failure, the appropriate error code shall be returned, and the response message content may contain additional error information.

6.1.4.10 Operation getSchemaByName

The API Consumer uses this operation to get the schema by name.

```
@startuml
autonumber
Participant "API Consumer" as Consumer
Participant "API Producer" as Producer
Consumer ->> Producer: GET .../schemas/{schemaName}/content
Producer -->> Consumer: 200 OK
@enduml
```

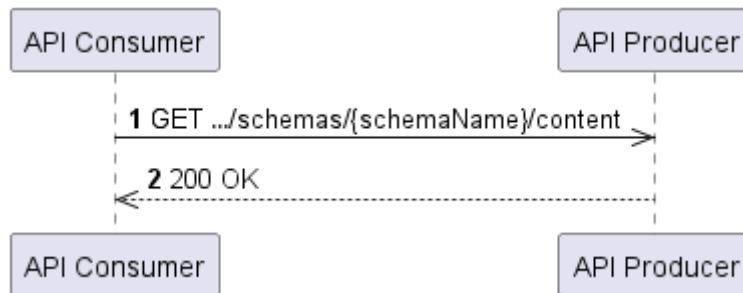


Figure 6.1.4.10-3: getSchemaByName Operation

The service operation is as follows:

1. The API Consumer shall send an HTTP GET request to the API Producer. The target URI shall identify the resource (.../schemas/{schemaName}/content) to discover the topology model schema.
2. The API Producer shall return the HTTP GET response. On success, "200 OK" shall be returned and the payload body of the GET response shall contain the obtained resource representation. On failure, the appropriate error code shall be returned, and the response message content may contain additional error information.

6.1.4.11 Operation getUserDefinedSchema

The API Consumer uses this operation to discover the available user defined schemas.

```
@startuml
autonumber
Participant "API Consumer" as Consumer
Participant "API Producer" as Producer
Consumer ->> Producer: GET .../user-defined-schemas
Producer -->> Consumer: 200 OK (UserDefinedSchemas)
@enduml
```

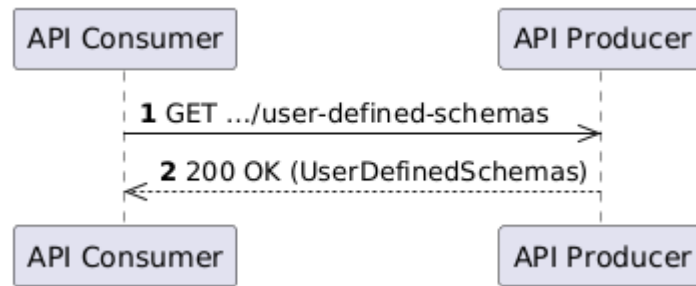


Figure 6.1.4.11-4: getUserDefinedSchema Operation

The service operation is as follows:

1. The API Consumer shall send an HTTP GET request to the API Producer. The target URI shall identify the resource (.../user-defined-schemas) to discover the available user defined schemas.
2. The API Producer shall return the HTTP GET response. On success, "200 OK" shall be returned and the message content shall carry a list of the available user defined schemas. On failure, the appropriate error code shall be returned, and the response message content may contain additional error information.

6.1.4.12 Operation createUserDefinedSchema

The API Consumer uses this operation to create new user defined schemas.

```

@startuml
autonumber
Participant "API Consumer" as Consumer
Participant "API Producer" as Producer
Consumer ->> Producer: POST .../user-defined-schemas (MultipartFile)
Producer -->> Consumer: 201 Created (UserDefinedSchema)
@enduml
  
```

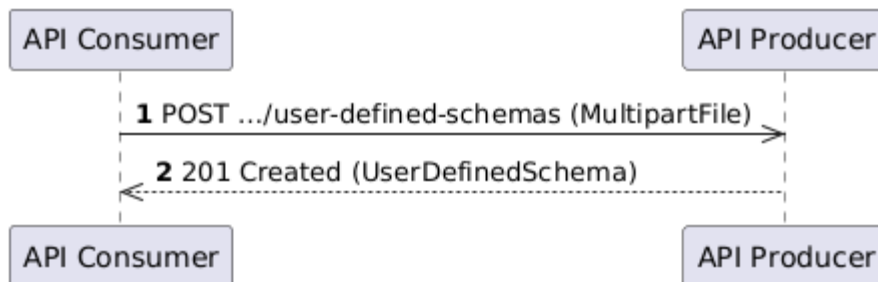


Figure 6.1.4.12-5: createUserDefinedSchema Operation

The service operation is as follows:

1. The API Consumer shall send an HTTP POST request to the API Producer. The target URI shall identify the resource (.../user-defined-schemas) under which the new user defined schema is requested to be created. The message content shall carry a MultipartFile structure.
2. The API Producer shall generate the user defined schema name and construct the URI for the created resource. The API Producer shall return the HTTP POST response. On success, "201 Created" shall be returned. The message shall carry a UserDefinedSchema structure that represents the new resource, and the "Location" header shall contain the URI of the created resource. On failure, the appropriate error code shall be returned, and the response message content may contain additional error information.

6.1.4.13 Operation deleteUserDefinedSchema

The API Consumer uses this operation to delete a user defined schema.

NOTE: Only user defined schemas can be deleted by the TE&IV API Consumer.

```
@startuml
autonumber
Participant "API Consumer" as Consumer
Participant "API Producer" as Producer
Consumer ->> Producer: DELETE ../user-defined-schemas/{schemaName}
Producer -->> Consumer: 204 No Content
@enduml
```

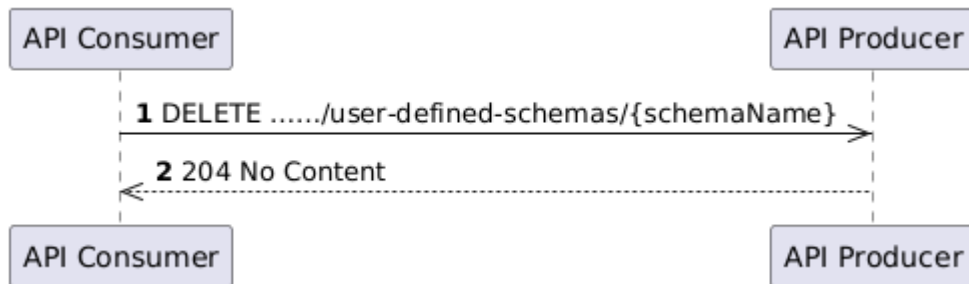


Figure 6.1.4.13-6: deleteUserDefinedSchema Operation

The service operation is as follows:

1. The API Consumer shall send an HTTP DELETE request to the API Producer. The target URI shall identify the resource (../user-defined-schemas/{schemaName}) to delete the user defined schema.
2. The API Producer shall return the HTTP DELETE response. On success, "204 No Content" shall be returned. On failure, the appropriate error code shall be returned, and the response message content may contain additional error information.

6.1.4.14 Operation getUserDefinedSchemaByName

The API Consumer uses this operation to get the user defined schema by name.

```
@startuml
autonumber
Participant "API Consumer" as Consumer
Participant "API Producer" as Producer
Consumer ->> Producer: GET ../user-defined-schemas/{schemaName}/content
Producer -->> Consumer: 200 OK
@enduml
```

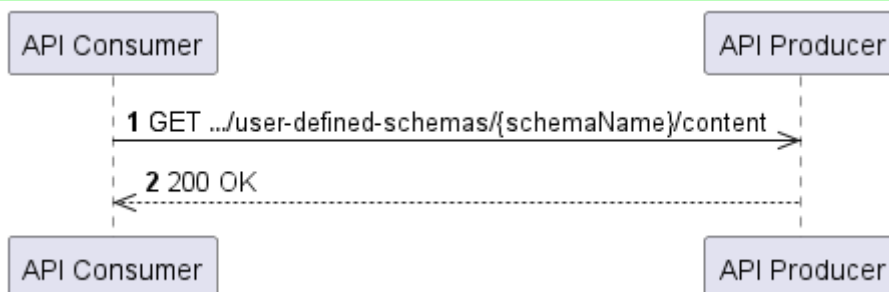


Figure 6.1.4.14-7: getUserDefinedSchemaByName Operation

The service operation is as follows:

1. The API Consumer shall send an HTTP GET request to the API Producer. The target URI shall identify the resource (.../user-defined-schemas/{schemaName}/content) to discover the content of the user defined schema by specific schema name.
2. The API Producer shall return the HTTP GET response. On success, “200 OK” shall be returned and the message content shall carry a string. On failure, the appropriate error code shall be returned, and the response message content may contain additional error information.

6.1.4.15 Operation updateClassifier

The API Consumer uses this operation to update or delete specified entities and/or relationships with classifiers.

```
@startuml
autonumber
Participant "API Consumer" as Consumer
Participant "API Producer" as Producer
Consumer ->> Producer: POST .../manage-classifiers (Classifier)
Producer -->> Consumer: 204 No Content
@enduml
```

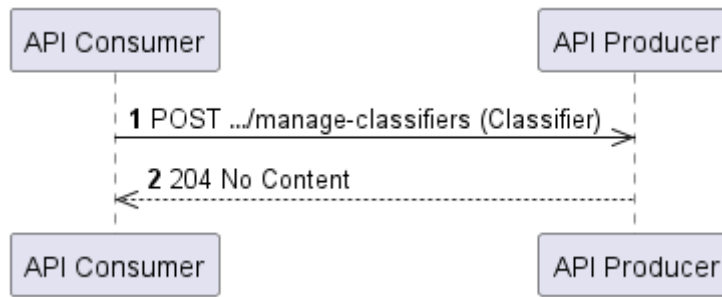


Figure 6.1.4.15-8: updateClassifier Operation

The service operation is as follows:

1. The API Consumer shall send an HTTP POST request to the API Producer. The target URI shall identify the resource (.../manage-classifiers) to update or delete the classifiers associated to the topology entities and topology relationships. The message content shall carry a Classifier structure.
2. The API Producer shall return the HTTP POST response. On success, “204 No Content” shall be returned. On failure, the appropriate error code shall be returned, and the response message content may contain additional error information.

6.1.4.16 Operation updateDecorator

The API Consumer uses this operation to update or delete specified entities and/or relationships with decorators.

```
@startuml
autonumber
Participant "API Consumer" as Consumer
Participant "API Producer" as Producer
Consumer ->> Producer: POST .../manage-decorators (Decorator)
Producer -->> Consumer: 204 No Content
@enduml
```



Figure 6.1.4.16-9: updateDecorator Operation

The service operation is as follows:

1. The API Consumer shall send an HTTP POST request to the API Producer. The target URI shall identify the resource (.../manage-decorators) to update or delete the decorators associated to the topology entities and topology relationship. The message content shall carry a Decorator structure.
2. The API Producer shall return the HTTP POST response. On success, “204 No Content ” shall be returned. On failure, the appropriate error code shall be returned, and the response message content may contain additional error information.

6.1.5 Resources

6.1.5.1 Overview

The following clause defines the resources for the Topology inventory API.

6.1.5.2 Resource: "All topology domains"

6.1.5.2.1 Description

The resource represents the available topology domains. Only the methods defined in clause 6.1.5.2.3 shall be supported by this resource.

6.1.5.2.2 Resource Definition

Resource URI: {apiRoot}/topology-inventory/<apiVersion>/domains

The resource URI variables supported by the resource are defined in Table 6.1.5.2.2-1.

Table 6.1.5.2.2-1: Resource URI variables for the resource

Name	Definition
apiRoot	See clause 5.2.
apiVersion	See clause 6.1.2.

6.1.5.2.3 Resource Standard Methods

6.1.5.2.3.1 GET

This method shall support the URI query parameters specified in table 6.1.5.2.3.1-1, URI header parameters specified in 6.1.5.2.3.1-2, the request data structure specified in the table 6.1.5.2.3.1-3 and the response data structure and response code specified in the table 6.1.5.2.3.1-4.

Table 6.1.5.2.3.1-1: URI query parameters supported by the GET method on this resource.

Name	Data type	P	Cardinality	Description
offsetParam	integer	O	0..1	This query parameter allows you to omit a specified number of entries before the beginning of the result set for pagination.
limitParam	integer	O	0..1	The query parameter provides to limit the number of entries returned for a request for pagination.

Table 6.1.5.2.3.1-2: URI header parameters supported by the GET method on this resource.

Name	Data type	P	Cardinality	Description
Accept	string	M	1	This header is used to specify response media types that are acceptable. The default type acceptable for this header is "application/json"

Table 6.1.5.2.3.1-3: Data structures supported by the GET request body on this resource.

Data Type	P	Cardinality	Description
N/A			There is no object in the message content of the GET request

Table 6.1.5.2.3.1-4: Data structures supported by the GET response body on this resource.

Data Type	P	Cardinality	Response codes	Description
Domains	M	1	200 OK	The operation was successful. The message content of the GET response carries a domains structure.
ProblemDetails	O	0..1	4xx/5xx	The operation has failed, and the message content may contain Problem description details.

6.1.5.2.4 Resource Custom Operations

None.

6.1.5.3 Resource: "All topology entity types in a topology domain"

6.1.5.3.1 Description

The resource represents the available topology entity types in a topology domain. Only the methods defined in clause 6.1.5.3.3 shall be supported by this resource.

6.1.5.3.2 Resource Definition

Resource URI: {apiRoot}/topology-inventory/<apiVersion>/domains/{domainName}/entity-types

The resource URI variables supported by the resource are defined in Table 6.1.5.3.2-1.

Table 6.1.5.3.2-1: Resource URI variables for the resource

Name	Definition
apiRoot	See clause 5.2 .
apiVersion	See clause 6.1.2.
domainName	Name of the Topology Domain.

6.1.5.3.3 Resource Standard Methods

6.1.5.3.3.1 GET

This method shall support the URI query parameters specified in table 6.1.5.3.3.1-1, URI header parameters specified in 6.1.5.3.3.1-2, the request data structure specified in the table 6.1.5.3.3.1-3 and the response data structure and response code specified in the table 6.1.5.3.3.1-4.

Table 6.1.5.3.3.1-1: URI query parameters supported by the GET method on this resource.

Name	Data type	P	Cardinality	Description
offsetParam	integer	O	0..1	This query parameter allows you to omit a specified number of entries before the beginning of the result set for pagination.
limitParam	integer	O	0..1	The query parameter provides to limit the number of entries returned for a request for pagination.

Table 6.1.5.3.3.1-2: URI header parameters supported by the GET method on this resource.

Name	Data type	P	Cardinality	Description
Accept	string	M	1	This header is used to specify response media types that are acceptable. The default type acceptable for this header is "application/json"

Table 6.1.5.3.3.1-3: Data structures supported by the GET request body on this resource.

Data Type	P	Cardinality	Description
N/A			There is no object in the message content of the GET request

Table 6.1.5.3.3.1-4: Data structures supported by the GET response body on this resource.

Data Type	P	Cardinality	Response codes	Description
EntityTypes	M	1	200 OK	The operation was successful. The message content of the GET response carries a EntityTypes structure.
ProblemDetails	O	0..1	4xx/5xx	The operation has failed, and the message content may contain Problem description details.

6.1.5.3.4 Resource Custom Operations

None.

6.1.5.4 Resource: "All instances of a topology entity type in a topology domain"

6.1.5.4.1 Description

The resource represents all the available instances of a topology entity types in a topology domain. Only the methods defined in clause 6.1.5.4.3 shall be supported by this resource.

6.1.5.4.2 Resource Definition

Resource URI:

{apiRoot}/topology-inventory/<apiVersion>/domains/{domainName}/entity-types/{entityTypeName}/entities

The resource URI variables supported by the resource are defined in Table 6.1.5.4.2-1.

Table 6.1.5.4.2-1: Resource URI variables for the resource

Name	Definition
apiRoot	See clause 5.2 .
apiVersion	See clause 6.1.2.
domainName	Name of the Topology Domain.
entityTypeName	Name of a topology entity in a topology domain.

6.1.5.4.3 Resource Standard Methods

6.1.5.4.3.1 GET

This method shall support the URI query parameters specified in table 6.1.5.4.3.1-1, URI header parameters specified in table 6.1.5.4.3.1-2 the request data structure specified in the table 6.1.5.4.3.1-3 and the response data structure and response code specified in the table 6.1.5.4.3.1-4.

Table 6.1.5.4.3.1-1: URI query parameters supported by the GET method on this resource.

Name	Data type	P	Cardinality	Description
targetFilter	string	O	0..1	This query parameter specifies the entity type and attributes to be returned in the REST response.
scopeFilter	string	O	0..1	This query parameter specifies the attributes to match on for specific Topology Entities for which the data is to be produced.
offsetParam	integer	O	0..1	This query parameter allows you to omit a specified number of entries before the beginning of the result set for pagination.
limitParam	integer	O	0..1	The query parameter provides to limit the number of entries returned for a request for pagination.

Table 6.1.5.4.3.1-2: URI header parameters supported by the GET method on this resource.

Name	Data type	P	Cardinality	Description
Accept	string	M	1	This header is used to specify response media types that are acceptable. The default type acceptable for this header is "application/yang.data+json"

Table 6.1.5.4.3.1-3: Data structures supported by the GET request body on this resource.

Data Type	P	Cardinality	Description
N/A			There is no object in the message content of the GET request

Table 6.1.5.4.3.1-4: Data structures supported by the GET response body on this resource.

Data Type	P	Cardinality	Response codes	Description
Entities	M	1	200 OK	The operation was successful. The message content of the GET response carries an Entities structure.
ProblemDetails	O	0..1	4xx/5xx	The operation has failed, and the message content may contain Problem description details.

6.1.5.4.4 Resource Custom Operations

None.

6.1.5.5 Resource: "Individual instance of a topology entity type in a topology domain"

6.1.5.5.1 Description

The resource represents the individual available instances of a topology entity types in a topology domain. Only the methods defined in clause 6.1.5.5.3 shall be supported by this resource.

6.1.5.5.2 Resource Definition

Resource URI:

{apiRoot}/topology-inventory/<apiVersion>/domain/{domainName}/entity-types/{entityTypeName}/entities/{entityId}

The resource URI variables supported by the resource are defined in Table 6.1.5.5.2-1.

Table 6.1.5.5.2-1: Resource URI variables for the resource

Name	Definition
apiRoot	See clause 5.2 .
apiVersion	See clause 6.1.2.
domainName	Name of the Topology Domain.
entityTypeName	Name of a topology entity in a topology domain.
entityId	Identifier of an individual topology entity.

6.1.5.5.3 Resource Standard Methods

6.1.5.5.3.1 GET

This method shall support the URI query parameters specified in table 6.1.5.5.3.1-1, URI header parameters specified in table 6.1.5.5.3.1-2, the request data structure specified in the table 6.1.5.5.3.1-3 and the response data structure and response code specified in the table 6.1.5.5.3.1-4.

Table 6.1.5.5.3.1-1: URI query parameters supported by the GET method on this resource.

Data Type	P	Cardinality	Description
N/A			There is no object in the message content of the GET request

Table 6.1.5.5.3.1-2: URI header parameters supported by the GET method on this resource.

Name	Data type	P	Cardinality	Description
Accept	string	M	1	This header is used to specify response media types that are acceptable. The default type acceptable for this header is "application/yang.data+json"

Table 6.1.5.5.3.1-3: Data structures supported by the GET request body on this resource.

Data Type	P	Cardinality	Description
N/A			There is no object in the message content of the GET request

Table 6.1.5.5.3.1-4: Data structures supported by the GET response body on this resource.

Data Type	P	Cardinality	Response codes	Description
EntityInstance	M	1	200 OK	The operation was successful. The message content of the GET response carries an EntityInstance which is an encapsulated object reference to the data model for schema definition of Topology Entity. (NOTE).
ProblemDetails	O	0..1	4xx/5xx	The operation has failed, and the message content may contain Problem description details.
NOTE: The attributes of a Topology Entity defined in the TE&IV Information Model [9] may be represented in multiple data modelling languages. Irrespective of the data modelling language specified in the Data Model specification [10], in the response, these attributes are encoded in JSON format. For example, attributes of the Topology Entity in the TE&IV Information Model [9] are represented in the YANG data modelling language. This YANG definition will be encoded as JSON using RFC 7951 and is encapsulated in this object.				

6.1.5.5.4 Resource Custom Operations

None.

6.1.5.6 Resource: "All relationships of an instance of a topology entity type in a topology domain"

6.1.5.6.1 Description

The resource represents the relationships of an instance of a topology entity types in a topology domain. Only the methods defined in clause 6.1.5.6.3 shall be supported by this resource.

6.1.5.6.2 Resource Definition

Resource URI:

{apiRoot}/topology-inventory/<apiVersion>/domains/{domainName}/entity-types/{entityTypeName}/entities/{entityId}/relationships

The resource URI variables supported by the resource are defined in Table 6.1.5.6.2-1.

Table 6.1.5.6.2-1: Resource URI variables for the resource

Name	Definition
apiRoot	See clause 5.2 .
apiVersion	See clause 6.1.2.
domainName	Name of the Topology Domain.
entityTypeName	Name of a topology entity in a topology domain.
entityId	Identifier of an individual topology entity.

6.1.5.6.3 Resource Standard Methods

6.1.5.6.3.1 GET

This method shall support the URI query parameters specified in table 6.1.5.6.3.1-1, URI header parameters specified in table 6.1.5.6.3.1-2, the request data structure specified in the table 6.1.5.6.3.1-3 and the response data structure and response code specified in the table 6.1.5.6.3.1-4.

Table 6.1.5.6.3.1-1: URI query parameters supported by the GET method on this resource.

Name	Data type	P	Cardinality	Description
offsetParam	integer	O	0..1	This query parameter allows you to omit a specified number of entries before the beginning of the result set for pagination.
limitParam	integer	O	0..1	The query parameter provides to limit the number of entries returned for a request for pagination.
targetFilter	string	O	0..1	This query parameter specifies the entity type and relationship to be returned in the REST response.
scopeFilter	string	O	0..1	This query parameter specifies the attributes to match on for specific Topology Entity relationship for which the data is to be produced.

Table 6.1.5.6.3.1-2: URI header parameters supported by the GET method on this resource.

Name	Data type	P	Cardinality	Description
Accept	string	M	1	This header is used to specify response media types that are acceptable. The default type acceptable for this header is "application/yang.data+json"

Table 6.1.5.6.3.1-3: Data structures supported by the GET request body on this resource.

Data Type	P	Cardinality	Description
N/A			There is no object in the message content of the GET request

Table 6.1.5.6.3.1-4: Data structures supported by the GET response body on this resource.

Data Type	P	Cardinality	Response codes	Description
Relationships	M	1	200 OK	The operation was successful. The message content of the GET response carries a Relationships structure.
ProblemDetails	O	0..1	4xx/5xx	The operation has failed, and the message content may contain Problem description details.

6.1.5.6.4 Resource Custom Operations

None.

6.1.5.7 Resource: "All topology relationship types in a topology domain"

6.1.5.7.1 Description

The resource represents all the topology relationship types in a topology domain. Only the methods defined in clause 6.1.5.7.3 shall be supported by this resource.

6.1.5.7.2 Resource Definition

Resource URI:

{apiRoot}/topology-inventory/<apiVersion>/domains/{domainName}/relationship-types

The resource URI variables supported by the resource are defined in Table 6.1.5.7.2-1.

Table 6.1.5.7.2-1: Resource URI variables for the resource

Name	Definition
apiRoot	See clause 5.2 .
apiVersion	See clause 6.1.2.
domainName	Name of the Topology Domain.

6.1.5.7.3 Resource Standard Methods

6.1.5.7.3.1 GET

This method shall support the URI query parameters specified in table 6.1.5.7.3.1-1, URI header parameters specified in table 6.1.5.7.3.1-2, the request data structure specified in the table 6.1.5.7.3.1-3 and the response data structure and response code specified in the table 6.1.5.7.3.1-4.

Table 6.1.5.7.3.1-1: URI query parameters supported by the GET method on this resource.

Name	Data type	P	Cardinality	Description
offsetParam	integer	O	0..1	This query parameter allows you to omit a specified number of entries before the beginning of the result set for pagination.
limitParam	integer	O	0..1	The query parameter provides to limit the number of entries returned for a request for pagination.

Table 6.1.5.7.3.1-2: URI header parameters supported by the GET method on this resource.

Name	Data type	P	Cardinality	Description
Accept	string	M	1	This header is used to specify response media types that are acceptable. The default type acceptable for this header is "application/json"

Table 6.1.5.7.3.1-3: Data structures supported by the GET request body on this resource.

Data Type	P	Cardinality	Description
N/A			There is no object in the message content of the GET request

Table 6.1.5.7.3.1-4: Data structures supported by the GET response body on this resource.

Data Type	P	Cardinality	Response codes	Description
RelationshipTypes	M	1	200 OK	The operation was successful. The message content of the GET response carries a RelationshipTypes structure.
ProblemDetails	O	0..1	4xx/5xx	The operation has failed, and the message content may contain Problem description details.

6.1.5.7.4 Resource Custom Operations

None.

6.1.5.8 Resource: "All topology relationship instances of a topology relationship type"

6.1.5.8.1 Description

The resource represents all the topology relationship instances of a topology relationship type in a topology domain. Only the methods defined in clause 6.1.5.8.3 shall be supported by this resource.

6.1.5.8.2 Resource Definition

Resource URI:

{apiRoot}/topology-inventory/<apiVersion>/domains/{domainName}/relationship-types/{relationshipTypeName}/relationships

The resource URI variables supported by the resource are defined in Table 6.1.5.8.2-1.

Table 6.1.5.8.2-1: Resource URI variables for the resource

Name	Definition
apiRoot	See clause 5.2.
apiVersion	See clause 6.1.2.
domainName	Name of the Topology Domain.
relationshipTypeName	A topology relationship type name such as NF Deployment NF Relationship.

6.1.5.8.3 Resource Standard Methods

6.1.5.8.3.1 GET

This method shall support the URI query parameters specified in table 6.1.5.8.3.1-1, URI header parameters specified in table 6.1.5.8.3.1-2, the request data structure specified in the table 6.1.5.8.3.1-3 and the response data structure and response code specified in the table 6.1.5.8.3.1-4.

Table 6.1.5.8.3.1-1: URI query parameters supported by the GET method on this resource.

Name	Data type	P	Cardinality	Description
offsetParam	integer	O	0..1	This query parameter allows you to omit a specified number of entries before the beginning of the result set for pagination.
limitParam	integer	O	0..1	The query parameter provides to limit the number of entries returned for a request for pagination.
targetFilter	string	O	0..1	This query parameter specifies the entity type and attributes to be returned in the REST response.
scopeFilter	string	O	0..1	This query parameter specifies the attributes to match on for specific Topology Entities for which the data is to be produced.

Table 6.1.5.8.3.1-2: URI header parameters supported by the GET method on this resource.

Name	Data type	P	Cardinality	Description
Accept	string	M	1	This header is used to specify response media types that are acceptable. The default type acceptable for this header is "application/json"

Table 6.1.5.8.3.1-3: Data structures supported by the GET request body on this resource.

Data Type	P	Cardinality	Description
N/A			There is no object in the message content of the GET request.

Table 6.1.5.8.3.1-4: Data structures supported by the GET response body on this resource.

Data Type	P	Cardinality	Response codes	Description
Relationships	M	1	200 OK	The operation was successful. The message content of the GET response carries a Relationships structure.
ProblemDetails	O	0..1	4xx/5xx	The operation has failed, and the message content may contain Problem description details.

6.1.5.8.4 Resource Custom Operations

None.

6.1.5.9 Resource: "Individual topology relationship instance of a topology relationship type"

6.1.5.9.1 Description

The resource represents all the topology relationship instances of a topology relationship type in a topology domain. Only the methods defined in clause 6.1.5.9.3 shall be supported by this resource.

6.1.5.9.2 Resource Definition

Resource URI:

{apiRoot}/topology-inventory/<apiVersion>/domains/{domainName}/relationship-types/{relationshipTypeName}/relationships/{relationshipId}

The resource URI variables supported by the resource are defined in Table 6.1.5.9.2-1.

Table 6.1.5.9.2-1: Resource URI variables for the resource

Name	Definition
apiRoot	See clause 5.2 .
apiVersion	See clause 6.1.2.
domainName	Name of the Topology Domain.
relationshipTypeName	A topology relationship type name.
relationshipId	An identifier of an individual topology relationship instance.

6.1.5.9.3 Resource Standard Methods

6.1.5.9.3.1 GET

This method shall support the URI query parameters specified in table 6.1.5.9.3.1-1, URI header parameters specified in table 6.1.5.9.3.1-2, the request data structure specified in the table 6.1.5.9.3.1-3 and the response data structure and response code specified in the table 6.1.5.9.3.1-4.

Table 6.1.5.9.3.1-1: URI query parameters supported by the GET method on this resource.

Data Type	P	Cardinality	Description
N/A			There is no object in the message content of the GET request.

Table 6.1.5.9.3.1-2: URI header parameters supported by the GET method on this resource.

Name	Data type	P	Cardinality	Description
Accept	string	M	1	This header is used to specify response media types that are acceptable. The default type acceptable for this header is "application/yang.data+json".

Table 6.1.5.9.3.1-3: Data structures supported by the GET request body on this resource.

Data Type	P	Cardinality	Description
N/A			There is no object in the message content of the GET request.

Table 6.1.5.9.3.1-4: Data structures supported by the GET response body on this resource.

Data Type	P	Cardinality	Response codes	Description
Relationship	M	1	200 OK	The operation was successful. The message content of the GET response carries a Relationship instance which is an encapsulated object reference to the data model for schema definition of Topology Relationship. (NOTE).
ProblemDetails	O	0..1	4xx/5xx	The operation has failed, and the message content may contain Problem description details.
NOTE: The attributes of a Topology Relationship defined in the TE&IV Information Model [9] may be represented in multiple data modelling languages. Irrespective of the data modelling language specified in the Data Model specification [10], in the response, these attributes are encoded in JSON format. For example, attributes of the Topology Relationship in the TE&IV Information Model [9] are represented in the YANG data modelling language. This YANG definition will be encoded as JSON using RFC 7951 and is encapsulated in this object.				

6.1.5.9.4 Resource Custom Operations

None.

6.1.5.10 Resource: "All topology model schemas"

6.1.5.10.1 Description

The resource represents all the available topology schema. Only the methods defined in clause 6.1.5.10.3 shall be supported by this resource.

6.1.5.10.2 Resource Definition

Resource URI: {apiRoot}/topology-inventory/<apiVersion>/schemas

The resource URI variables supported by the resource are defined in Table 6.1.5.10.2-1.

Table 6.1.5.10.2-1: Resource URI variables for the resource

Name	Definition
apiRoot	See clause 5.2.
apiVersion	See clause 6.1.2.

6.1.5.10.3 Resource Standard Methods

6.1.5.10.3.1 GET

This method shall support the URI query parameters specified in table 6.1.5.10.3.1-1, URI header parameters specified in 6.1.5.10.3.1-2, the request data structure specified in the table 6.1.5.10.3.1-3 and the response data structure and response code specified in the table 6.1.5.10.3.1-4.

Table 6.1.5.10.3.1-1: URI query parameters supported by the GET method on this resource.

Name	Data type	P	Cardinality	Description
domain	string	O	0..1	This query parameter allows you to specify the desired domain
offsetParam	integer	O	0..1	This query parameter allows you to omit a specified number of entries before the beginning of the result set for pagination.
limitParam	integer	O	0..1	The query parameter provides to limit the number of entries returned for a request for pagination.

Table 6.1.5.10.3.1-2: URI header parameters supported by the GET method on this resource.

Name	Data type	P	Cardinality	Description
Accept	string	M	1	This header is used to specify response media types that are acceptable. The default type acceptable for this header is "application/json"

Table 6.1.5.10.3.1-3: Data structures supported by the GET request body on this resource.

Data Type	P	Cardinality	Description
N/A			There is no object in the message content of the GET request

Table 6.1.5.10.3.1-4: Data structures supported by the GET response body on this resource.

Data Type	P	Cardinality	Response codes	Description
Schemas	M	1	200 OK	The operation was successful. The message content of the GET response carries a Schemas structure.
ProblemDetails	O	0..1	4xx/5xx	The operation has failed, and the message content may contain Problem description details.

6.1.5.10.4 Resource Custom Operations

None.

6.1.5.11 Resource: "Individual model schema content"

6.1.5.11.1 Description

The resource represents the content of a schema instance. Only the methods defined in clause 6.1.5.11.3 shall be supported by this resource.

6.1.5.11.2 Resource Definition

Resource URI: **{apiRoot}/topology-inventory/<apiVersion>/schemas/{schemaName}/content**

The resource URI variables supported by the resource are defined in Table 6.1.5.11.2-1.

Table 6.1.5.11.2-1: Resource URI variables for the resource

Name	Definition
apiRoot	See clause 5.2.
apiVersion	See clause 6.1.2.
schemaName	Name of the topology schema

6.1.5.11.3 Resource Standard Methods

6.1.5.11.3.1 GET

This method shall support the URI query parameters specified in table 6.1.5.11.3.1-1, URI header parameters specified in 6.1.5.11.3.1-2, the request data structure specified in the table 6.1.5.11.3.1-3 and the response data structure and response code specified in the table 6.1.5.11.3.1-4.

Table 6.1.5.11.3.1-1: URI query parameters supported by the GET method on this resource.

Name	Data type	P	Cardinality	Description
N/A				

Table 6.1.5.11.3.1-2: URI header parameters supported by the GET method on this resource.

Name	Data type	P	Cardinality	Description
Accept	string	M	1	This header is used to specify response media types that are acceptable. The default type acceptable for this header is "text/plain"

Table 6.1.5.11.3.1-3: Data structures supported by the GET request body on this resource.

Data Type	P	Cardinality	Description
N/A			

Table 6.1.5.11.3.1-4: Data structures supported by the GET response body on this resource.

Data Type	P	Cardinality	Response codes	Description
string	M	1	200 OK	The operation was successful. The payload body of the GET response carries the Schema content.
ProblemDetails	O	0..1	4xx/5xx	The operation has failed, and the message content may contain Problem description details.

6.1.5.11.4 Resource Custom Operations

None.

6.1.5.12 Resource: "All user defined schemas"

6.1.5.12.1 Description

The resource represents all the available user defined schema. Only the methods defined in clause 6.1.5.12.3 shall be supported by this resource.

6.1.5.12.2 Resource Definition

Resource URI: **{apiRoot}/topology-inventory/<apiVersion>/user-defined-schemas**

The resource URI variables supported by the resource are defined in Table 6.1.5.12.2-1.

Table 6.1.5.12.2-1: Resource URI variables for the resource

Name	Definition
apiRoot	See clause 5.2.
apiVersion	See clause 6.1.2.

6.1.5.12.3 Resource Standard Methods

6.1.5.12.3.1 GET

This method shall support the URI query parameters specified in table 6.1.5.12.3.1-1, URI header parameters specified in 6.1.5.12.3.1-2, the request data structure specified in the table 6.1.5.12.3.1-3 and the response data structure and response code specified in the table 6.1.5.12.3.1-4.

Table 6.1.5.12.3.1-1: URI query parameters supported by the GET method on this resource.

Name	Data type	P	Cardinality	Description
offsetParam	integer	O	0..1	This query parameter allows you to omit a specified number of entries before the beginning of the result set for pagination.
limitParam	integer	O	0..1	The query parameter provides to limit the number of entries returned for a request for pagination.

Table 6.1.5.12.3.1-2: URI header parameters supported by the GET method on this resource.

Name	Data type	P	Cardinality	Description
Accept	string	M	1	This header is used to specify response media types that are acceptable. The default type acceptable for this header is "application/json"

Table 6.1.5.12.3.1-3: Data structures supported by the GET request body on this resource.

Data Type	P	Cardinality	Description
N/A			There is no object in the message content of the GET request

Table 6.1.5.12.3.1-4: Data structures supported by the GET response body on this resource.

Data Type	P	Cardinality	Response codes	Description
UserDefinedSchemas	M	1	200 OK	The operation was successful. The payload body of the GET response carries a UserDefinedSchemas structure.
ProblemDetails	O	0..1	4xx/5xx	The operation has failed, and the message content may contain Problem description details.

6.1.5.12.3.2 POST

This method shall support the URI query parameters specified in table 6.1.5.12.3.2-1, URI header parameters specified in 6.1.5.12.3.2-2, the request data structure specified in the table 6.1.5.12.3.2-3 and the response data structure and response code specified in the table 6.1.5.12.3.2-4.

Table 6.1.5.12.3.2-1: URI query parameters supported by the POST method on this resource.

Name	Data type	P	Cardinality	Description
N/A				

Table 6.1.5.12.3.2-2: URI header parameters supported by the POST method on this resource.

Name	Data type	P	Cardinality	Description
Accept	string	M	1	This header is used to specify response media types that are acceptable. The default type acceptable for this header is "application/json"
Content-Type	string	M	1	This header is used to specify resource media types that are acceptable. The default type acceptable for this header is "multipart/form-data" as per RFC 7578 [x]

Table 6.1.5.12.3.2-3: Data structures supported by the POST request body on this resource.

Data Type	P	Cardinality	Description
MultipartFile	M	1	Multipart file containing the user defined schema to be created.

Table 6.1.5.12.3.2-4: Data structures supported by the POST response body on this resource

Data Type	P	Cardinality	Response codes	Description
UserDefinedSchema	M	1	201 Created	The operation was successful. The message content of the POST response carries a UserDefinedSchema structure.
ProblemDetails	O	0..1	4xx/5xx	The operation has failed, and the message content may contain Problem description details.

Table 6.1.5.12.3.2-5: Headers supported by the 201 Response code on this resource

Data Type	P	Cardinality	Description
Location	M	1	Contains the URI of the newly created "Individual User Defined Schema" resource as defined in clause 6.1.5.12

6.1.5.12.4 Resource Custom Operations

None.

6.1.5.13 Resource: "Individual user defined schema"

6.1.5.13.1 Description

The resource represents an individual user defined schema. Only the methods defined in clause 6.1.5.13.3 shall be supported by this resource.

6.1.5.13.2 Resource Definition

Resource URI: {apiRoot}/topology-inventory/<apiVersion>/user-defined-schemas/{schemaName}

The resource URI variables supported by the resource are defined in Table 6.1.5.13.2-1.

Table 6.1.5.13.2-1: Resource URI variables for the resource

Name	Definition
apiRoot	See clause 5.2.
apiVersion	See clause 6.1.2.
schemaName	Name of user defined schema

6.1.5.13.3 Resource Standard Methods

6.1.5.13.3.1 DELETE

This method shall support the URI query parameters specified in table 6.1.5.13.3.1-1, URI header parameters specified in 6.1.5.13.3.1-2, the request data structure specified in the table 6.1.5.13.3.1-3 and the response data structure and response code specified in the table 6.1.5.13.3.1-4.

Table 6.1.5.13.3.1-1: URI query parameters supported by the DELETE method on this resource.

Name	Data type	P	Cardinality	Description
N/A				

Table 6.1.5.13.3.1-2: URI header parameters supported by the DELETE method on this resource.

Name	Data type	P	Cardinality	Description
Accept	string	M	1	This header is used to specify response media types that are acceptable. The default type acceptable for this header is "application/json"

Table 6.1.5.13.3.1-3: Data structures supported by the DELETE request body on this resource.

Data Type	P	Cardinality	Description
N/A			

Table 6.1.5.13.3.1-4: Data structures supported by the DELETE response body on this resource.

Data Type	P	Cardinality	Response codes	Description
N/A			204 No Content	The operation was successful
ProblemDetails	O	0..1	4xx/5xx	The operation has failed, and the message content may contain Problem description details.

6.1.5.13.4 Resource Custom Operations

None.

6.1.5.14 Resource: "Individual user defined schema content"

6.1.5.14.1 Description

The resource represents the content of a user defined schema instance. Only the methods defined in clause 6.1.5.14.3 shall be supported by this resource.

6.1.5.14.2 Resource Definition

Resource URI: **{apiRoot}/topology-inventory/<apiVersion>/user-defined-schemas/{schemaName}/content**

The resource URI variables supported by the resource are defined in Table 6.1.5.14.2-1.

Table 6.1.5.14.2-1: Resource URI variables for the resource

Name	Definition
apiRoot	See clause 5.2.
apiVersion	See clause 6.1.2.
schemaName	Name of the topology schema

6.1.5.14.3 Resource Standard Methods

6.1.5.14.3.1 GET

This method shall support the URI query parameters specified in table 6.1.5.14.3.1-1, URI header parameters specified in 6.1.5.14.3.1-2, the request data structure specified in the table 6.1.5.14.3.1-3 and the response data structure and response code specified in the table 6.1.5.14.3.1-4.

Table 6.1.5.14.3.1-1: URI query parameters supported by the GET method on this resource.

Name	Data type	P	Cardinality	Description
N/A				

Table 6.1.5.14.3.1-2: URI header parameters supported by the GET method on this resource.

Name	Data type	P	Cardinality	Description
Accept	string	M	1	This header is used to specify response media types that are acceptable. The default type acceptable for this header is "text/plain"

Table 6.1.5.14.3.1-3: Data structures supported by the GET request body on this resource.

Data Type	P	Cardinality	Description
N/A			

Table 6.1.5.14.3.1-4: Data structures supported by the GET response body on this resource.

Data Type	P	Cardinality	Response codes	Description
string	M	1	200 OK	The operation was successful. The payload body of the GET response carries the user defined schema content.
ProblemDetails	O	0..1	4xx/5xx	The operation has failed, and the message content may contain Problem description details.

6.1.5.14.4 Resource Custom Operations

None.

6.1.6 Custom operations without associated resources

6.1.6.1 Overview

The following clause defines the custom operations for the Topology Inventory API.

6.1.6.2 Operation: manage-classifiers

6.1.6.2.1 Description

This custom operation allows the API consumer to manage (add/remove) classifiers on entities and/or relationships.

6.1.6.2.2 Operation Definition

Custom Operation URI: **{apiRoot}/topology-inventory/<apiVersion>/manage-classifiers**

This operation shall support the URI header parameters specified in 6.1.6.2.2.1-1, the request data structure specified in the table 6.1.6.2.2.1-2 and the response data structure and response code specified in the table 6.1.6.2.2.1-3.

Table 6.1.6.2.2.1-1: URI header parameters supported by the POST method on this resource.

Name	Data type	P	Cardinality	Description
Accept	string	M	1	This header is used to specify response media types that are acceptable. The default type acceptable for this header is "application/json"
Content-Type	string	M	1	This header is used to specify resource media types that are acceptable. The default type acceptable for this header is "application/json"

Table 6.1.6.2.2.1-2: Data structures supported by the POST request body on this resource.

Data Type	P	Cardinality	Description
Classifier	M	1	Classifier Schema to update or delete the classifiers

Table 6.1.6.2.2.1-3: Data structures supported by the POST response body on this resource.

Data Type	P	Cardinality	Response codes	Description
N/A			204 No Content	The operation has succeeded. There is no message content
ProblemDetails	O	0..1	4xx/5xx	The operation has failed, and the message content may contain Problem description details.

6.1.6.3 Operation: manage-decorators

6.1.6.3.1 Description

This custom operation allows the API consumer to manage (add/remove) decorators on entities and/or relationships.

6.1.6.3.2 Operation Definition

Custom Operation URI: **{apiRoot}/topology-inventory/<apiVersion>/manage-decorators**

This operation shall support the URI header parameters specified in 6.1.6.3.2.1-1, the request data structure specified in the table 6.1.6.3.2.1-2 and the response data structure and response code specified in the table 6.1.6.3.2.1-3.

Table 6.1.6.3.2.1-1: URI header parameters supported by the POST method on this resource.

Name	Data type	P	Cardinality	Description
Accept	string	M	1	This header is used to specify response media types that are acceptable. The default type acceptable for this header is "application/json"
Content-Type	string	M	1	This header is used to specify resource media types that are acceptable. The default type acceptable for this header is "application/json"

Table 6.1.6.3.2.1-2: Data structures supported by the POST request body on this resource.

Data Type	P	Cardinality	Description
Decorator	M	1	Decorator Schema to update or delete the decorators

Table 6.1.6.3.2.1-3: Data structures supported by the POST response body on this resource.

Data Type	P	Cardinality	Response codes	Description
N/A			204 No Content	The operation has succeeded. There is no message content.
ProblemDetails	O	0..1	4xx/5xx	The operation has failed, and the message content may contain Problem description details.

6.1.7 Notifications

None.

6.1.8 Data model

6.1.8.1 Structured data types

6.1.8.1.1 Overview

The following clauses define the structured data types and their attributes to be used by the service API.

6.1.8.1.2 Data type: Domains

The Domains data type represents domains. It contains the attributes defined in table 6.1.8.1.2-1.

Table 6.1.8.1.2-1: Definition of type Domains.

Attribute Name	Data type	P	Cardinality	Description
items	array(Domain)	M	0..N	A list of domains.

6.1.8.1.3 Data type: Domain

The Domain data type represents the properties of a topology domain. It contains the attributes defined in table 6.1.8.1.3-1.

Table 6.1.8.1.3-1: Definition of type Domain.

Attribute Name	Data type	P	Cardinality	Description
domainName	string	M	1	A name of the topology domain.
entityTypes	Href	M	1	A reference to entity types of the Topology Entity.
relationshipTypes	Href	M	1	A reference to relationship types of the Topology Relationship.

6.1.8.1.4 Data type: EntityTypes

The EntityTypes data type represents entity type within a topology domain. It contains the attributes defined table 6.1.8.1.4-1.

Table 6.1.8.1.4-1: Definition of type EntityTypes.

Attribute Name	Data type	P	Cardinality	Description
items	array(EntityType)	M	1..N	An entity type within a topology domain.

6.1.8.1.5 Data type: EntityType

The EntityType data type represents a type within a topology domain.

Table 6.1.8.1.5-1: Definition of type EntityType.

Attribute Name	Data type	P	Cardinality	Description
entityTypeName	string	M	1	A name of a topology Entity Type.
entities	Href	M	1	A reference to topology entities of the specific topology Entity Type

6.1.8.1.6 Data type: Entities

The Entities data type represents all instances of a specific topology entity type within a topology domain. It contains the attributes defined in table 6.1.8.1.6-1.

Table 6.1.8.1.6-1: Definition of type Entities.

Attribute Name	Data type	P	Cardinality	Description
items	array(object)	M	1..N	Topology Entity instances encapsulated as array of objects and refers to the data model for schema definition of Topology Entity. (NOTE).
NOTE: The attributes of a Topology Entity defined in the TE&IV Information Model [9] may be represented in multiple data modelling languages. Irrespective of the data modelling language specified in the Data Model specification [10], in the response, these attributes are encoded in JSON format. For example, attributes of the Topology Entity in the TE&IV Information Model [9] are represented in the YANG data modelling language. This YANG definition will be encoded as JSON using RFC 7951 and is encapsulated in this object.				

6.1.8.1.7 Data type: Relationships

The Relationships data type represents relationships information of an instance of a topology entity type within a topology domain. It contains the attributes defined in table 6.1.8.1.7-1.

Table 6.1.8.1.7-1: Definition of type Relationships.

Attribute Name	Data type	P	Cardinality	Description
items	array(object)	M	1..N	Encapsulated object reference to the data model for schema definition of Topology Relationship. (NOTE).
NOTE: The attributes of the Topology Relationship of a Topology Entity defined in the TE&IV Information Model [9] may be represented in multiple data modelling languages. Irrespective of the data modelling language specified in the Data Model specification [10], in the response, these attributes are encoded in JSON format. For example, attributes of the Topology Relationship of a Topology Entity in a TE&IV REL-RAN-Cloud domain in the TE&IV Information Model [9] are represented in the YANG data modelling language. This YANG definition will be encoded as JSON using RFC 7951 and is encapsulated in this object.				

6.1.8.1.8 Data type: RelationshipTypes

The Relationships data type represents all the relationships information of a topology entity type within a topology domain. It contains the attributes defined in table 6.1.8.1.8-1.

Table 6.1.8.1.8-1: Definition of type RelationshipTypes.

Attribute Name	Data type	P	Cardinality	Description
items	array(RelationshipType)	M	1..N	All the relationship type information within a topology domain.

6.1.8.1.9 Data type: RelationshipType

The relationship information in a topology domain. It contains the attributes defined in table 6.1.8.1.9-1.

Table 6.1.8.1.9-1: Definition of type RelationshipType.

Attribute Name	Data type	P	Cardinality	Description
relationshipTypeName	string	M	1	Name of a Topology Relationship.
relationships	Href	M	1	URI to the relationships of the Topology Relationship Type.

6.1.8.1.10 Data type: Href

This attribute provide reference to entity type and relationship type. It contains the attributes defined in table 6.1.8.1.10-1.

Table 6.1.8.1.10-1: Definition of type Href

Attribute Name	Data type	P	Cardinality	Description
href	string	M	1	The format of the string according to IETF RFC 3986 [6].

6.1.8.1.11 Data type: Schemas

The Schema data type represents a topology or user defined schema. It contains the attributes defined in table 6.1.8.1.11-1.

Table 6.1.8.1.11-1: Definition of type Schemas.

Attribute Name	Data type	P	Cardinality	Description
items	array(Schema)	M	1..N	List of schemas.

6.1.8.1.12 Data type: Schema

The Schema data type represents a topology model or user defined schema. It contains the attributes defined in table 6.1.8.1.12-1.

Table 6.1.8.1.12-1: Definition of type Schema.

Attribute Name	Data type	P	Cardinality	Description
name	string	M	1	Name of the schema
domain	string	M	0..1	Domain where schema is set.
revision	string	M	1	Revision date of the YANG schema, as defined in the module's revision statement.
content	Href	M	1	URI to the schema content. Data type is defined in clause 6.1.8.1.10.

6.1.8.1.13 Data type: UserDefinedSchemas

The UserDefinedSchema data type represents a user defined schema. It contains the attributes defined in table 6.1.8.1.13-1.

Table 6.1.8.1.13-1: Definition of type UserDefinedSchemas.

Attribute Name	Data type	P	Cardinality	Description
items	array(UserDefinedSchema)	M	1..N	List of user defined schemas.

6.1.8.1.14 Data type: UserDefinedSchema

The UserDefinedSchema data type represents a user defined schema. It contains the attributes defined in table 6.1.8.1.14-1.

Table 6.1.8.1.14-1: Definition of type UserDefinedSchema.

Attribute Name	Data type	P	Cardinality	Description
name	string	M	1	Name of the user defined schema
revision	string	M	1	Revision date of the YANG schema, as defined in the module's revision statement.
content	Href	M	1	URI to the schema content. Data type is defined in clause 6.1.8.1.10.

6.1.8.1.15 Data type: MultipartFile

The MultipartFile data type represents a Multipart File. It contains the attributes defined in table 6.1.8.1.15-1.

Table 6.1.8.1.15-1: Definition of type MultipartFile.

Attribute Name	Data type	P	Cardinality	Description
file	string	M	1	File containing a schema in YANG format.

6.1.8.1.16 Data type: Classifier

The Classifier data type represents a classifier. It contains the attributes defined in table 6.1.8.1.16-1.

Table 6.1.8.1.16-1: Definition of type Classifier.

Attribute Name	Data type	P	Cardinality	Description
operation	ENUM	M	1	Indicates whether the classifier is to be merged or deleted. Allowed values: merge, delete
classifiers	array(String)	M	1..N	The strings to be used as classifiers
entityIds	array(String)	M	0..N	Ids of the relationships to be classified
relationshipIds	array(String)	M	0..N	Ids of the relationships to be classified

6.1.8.1.17 Data type: Decorator

The Decorator data type represents a decorator. It contains the attributes defined in table 6.1.8.1.17-1.

Table 6.1.8.1.17-1: Definition of type Decorator.

Attribute Name	Data type	P	Cardinality	Description
operation	ENUM	M	1	Indicates whether the decorator is to be merged or deleted Allowed values: merge, delete
decorators	AttributeValuePair	M	1..N	The key-value pairs to be used as decorators. Allowed values: Any string, integer, or boolean
entityIds	array(String)	M	0..N	Ids of the entities to be decorated
relationshipIds	array(String)	M	0..N	Ids of the relationships to be decorated

6.1.8.2 Simple data types and enumerations

The following clause defines simple data types and enumerations that can be referenced from data structures defined in the previous clauses.

6.1.8.2.1 Simple data types

For this service API, no simple data types are defined in the present document

6.1.8.2.2 Enumerations

For this service API, no enumerations are defined in the present document.

6.1.9 Error Handling

6.1.9.1 General

For the Topology Inventory API, HTTP error responses shall be supported as specified in clause 4.8 of 3GPP TS 29.501 [3]. Protocol errors and application errors specified in table 5.2.7.2-1 of 3GPP TS 29.500 [2] shall be supported for an HTTP method if the corresponding HTTP status codes are specified as mandatory for that HTTP method in table 5.2.7.1-1 of 3GPP TS 29.500 [2].

In addition, the requirements in the following clauses are applicable for the Topology Inventory API.

6.1.9.2 Protocol Errors

No specific protocol errors are defined in the present document.

6.1.9.3 Application Errors

No additional application errors defined in the present document.

Annex A (normative): OpenAPI Specifications

A.1 Overview

This Annex formally specifies the RESTful Topology & Inventory API by defining OpenAPI documents in YAML format that comply with the OpenAPI 3.0.3 Specification [11].

The Open API specifications of the RESTful Topology & Inventory API provided in this annex are versioned as described in clause 5.2.

A.2 Topology & Inventory API

```
openapi: 3.0.3
info:
  title: 'Topology & Inventory API'
  version: 1.2.1
  description: |
    API for Topology and Inventory API.

    Topology and Inventory data is the information that represents entities
    in a telecommunications network and the relationships between them that
    provide insight into a particular aspect of the network of importance to
    specific use cases. Topology and Inventory data can be derived from
    inventory, configuration, or other data.

    Topology and Inventory supports several topology domains. A domain is a
    grouping of topology and inventory entities that handles topology and
    inventory data.

    Entities are enabling the modelling and storage of complex network
    infrastructure and relationships.

    A relationship is a bi-directional connection between two entities, one
```

of which is the originating side (A-side) and the other is the terminating side (B-side). The order of the sides matters since it defines the relationship itself which must be unique.

Classifier (also known as tag or label) permits the association of a well defined user specified string with an entity or relationship.

Decorators are user-defined attributes (key-value pairs) which can be applied to topology entities and relationships.

Topology and Inventory API provides the capabilities to fetch topology data. Using the filtering options, it is possible to define more specific query requests.

Copyright 2026 O-RAN ALLIANCE e.V.

Licensed under the Standards Collaboration
Copyright License Version 1.0 (the "License");
You may not use this file except in compliance with the License.
You may obtain a copy of the License at
<https://www.o-ran.org/sccl>

tags:

- name: Entities and relationships
description: Provides the capability to retrieve topology and inventory entities and relationships.
- name: Schemas
description: Schemas are defined in YANG modeling language. A group of Yang schemas makes the topology & inventory model, which represents topology & inventory entities, their attributes, and their relationships.
- name: User Defined Schemas
description: User defined schemas are defined in YANG modeling language.
- name: Classifiers
description: Provides the capability to update or remove user-defined keywords or tags on entities and relationships.
- name: Decorators
description: Provides the capability to update or remove user-defined values on entities and relationships.

externalDocs:

description: 'O-RAN.WG10.TS.TE&IV-API.0-R005-v04.00'
url: 'https://www.o-ran.org/specifications'

servers:

- url: '{apiRoot}/topology-inventory/v1'
variables:
apiRoot:
description: apiRoot as defined in clause 5.3 in O-RAN.WG10.TS.TE&IV-API.0
default: 'https://example.com'

paths:

```
/domains:
  get:
    description: Get all the available topology domains.
    tags:
      - Entities and relationships
    summary: Get all the available topology domains.
    operationId: getAllDomains
    parameters:
      - $ref: '#/components/parameters/acceptJsonInHeader'
      - $ref: '#/components/parameters/offsetParam'
      - $ref: '#/components/parameters/limitParam'
    responses:
      '200':
        description: OK
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/Domains'
      '400':
        $ref: '#/components/responses/400'
      '401':
```

```

    $ref: '#/components/responses/401'
  '403':
    $ref: '#/components/responses/403'
  '404':
    $ref: '#/components/responses/404'
  '406':
    $ref: '#/components/responses/406'
  '414':
    $ref: '#/components/responses/414'
  '429':
    $ref: '#/components/responses/429'
  '500':
    $ref: '#/components/responses/500'
  '502':
    $ref: '#/components/responses/502'
  '503':
    $ref: '#/components/responses/503'
/contexts/{domainName}/entity-types:
get:
  description: Get all the available topology entity types within a domain.
  tags:
    - Entities and relationships
  summary: Get all the available topology entity types within a domain.
  operationId: getTopologyEntityTypes
  parameters:
    - $ref: '#/components/parameters/acceptJsonInHeader'
    - $ref: '#/components/parameters/domainNameInPath'
    - $ref: '#/components/parameters/offsetParam'
    - $ref: '#/components/parameters/limitParam'
  responses:
    '200':
      description: OK
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/EntityTypes'
    '400':
      $ref: '#/components/responses/400'
    '401':
      $ref: '#/components/responses/401'
    '403':
      $ref: '#/components/responses/403'
    '404':
      $ref: '#/components/responses/404'
    '406':
      $ref: '#/components/responses/406'
    '414':
      $ref: '#/components/responses/414'
    '429':
      $ref: '#/components/responses/429'
    '500':
      $ref: '#/components/responses/500'
    '502':
      $ref: '#/components/responses/502'
    '503':
      $ref: '#/components/responses/503'
/contexts/{domainName}/entity-types/{entityTypeName}/entities:
get:
  description: Get all Topology Entity instances of a specific Topology Entity type.
  tags:
    - Entities and relationships
  summary: Get all Topology Entity instances of a specific Topology Entity type.
  operationId: getTopologyByEntityTypeName
  parameters:
    - $ref: '#/components/parameters/acceptJsonInHeader'
    - $ref: '#/components/parameters/domainNameInPath'
    - $ref: '#/components/parameters/entityTypeNameInPath'
    - $ref: '#/components/parameters/targetFilterOptionalInQuery'
    - $ref: '#/components/parameters/scopeFilterOptionalInQuery'
    - $ref: '#/components/parameters/offsetParam'

```

```

- $ref: '#/components/parameters/limitParam'
responses:
  '200':
    description: OK
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/Entities'
  '400':
    $ref: '#/components/responses/400'
  '401':
    $ref: '#/components/responses/401'
  '403':
    $ref: '#/components/responses/403'
  '404':
    $ref: '#/components/responses/404'
  '406':
    $ref: '#/components/responses/406'
  '414':
    $ref: '#/components/responses/414'
  '429':
    $ref: '#/components/responses/429'
  '500':
    $ref: '#/components/responses/500'
  '502':
    $ref: '#/components/responses/502'
  '503':
    $ref: '#/components/responses/503'

/domains/{domainName}/entity-types/{entityTypeName}/entities/{entityId}:
get:
  description: Get a specific Topology Entity instance of a Topology Entity type
  tags:
    - Entities and relationships
  summary: Get a specific Topology Entity instance of a Topology Entity type
  operationId: getTopologyById
  parameters:
    - $ref: '#/components/parameters/acceptYangJsonInHeader'
    - $ref: '#/components/parameters/domainNameInPath'
    - $ref: '#/components/parameters/entityTypeNameInPath'
    - $ref: '#/components/parameters/entityIdInPath'
  responses:
    '200':
      description: OK
      content:
        application/yang.data+json:
          schema:
            type: object
            description: Encapsulated object reference to the data model for schema definition of
Topology Entity
  '400':
    $ref: '#/components/responses/400'
  '401':
    $ref: '#/components/responses/401'
  '403':
    $ref: '#/components/responses/403'
  '404':
    $ref: '#/components/responses/404'
  '406':
    $ref: '#/components/responses/406'
  '414':
    $ref: '#/components/responses/414'
  '429':
    $ref: '#/components/responses/429'
  '500':
    $ref: '#/components/responses/500'
  '502':
    $ref: '#/components/responses/502'
  '503':
    $ref: '#/components/responses/503'

```

```

/domains/{domainName}/entity-types/{entityTypeName}/entities/{entityId}/relationships:
get:
  description: Get all relationships for a specific Topology Entity instance of a Topology Entity type
  tags:
    - Entities and relationships
  summary: Get all relationships for a specific Topology Entity instance of a Topology Entity type
  operationId: getAllRelationshipsForEntityId
  parameters:
    - $ref: '#/components/parameters/acceptYangJsonInHeader'
    - $ref: '#/components/parameters/domainNameInPath'
    - $ref: '#/components/parameters/entityTypeNameInPath'
    - $ref: '#/components/parameters/entityIdInPath'
    - $ref: '#/components/parameters/targetFilterOptionalInQuery'
    - $ref: '#/components/parameters/scopeFilterOptionalInQuery'
    - $ref: '#/components/parameters/offsetParam'
    - $ref: '#/components/parameters/limitParam'
  responses:
    '200':
      description: OK
      content:
        application/yang.data+json:
          schema:
            $ref: '#/components/schemas/Relationships'
    '400':
      $ref: '#/components/responses/400'
    '401':
      $ref: '#/components/responses/401'
    '403':
      $ref: '#/components/responses/403'
    '404':
      $ref: '#/components/responses/404'
    '406':
      $ref: '#/components/responses/406'
    '414':
      $ref: '#/components/responses/414'
    '429':
      $ref: '#/components/responses/429'
    '500':
      $ref: '#/components/responses/500'
    '502':
      $ref: '#/components/responses/502'
    '503':
      $ref: '#/components/responses/503'

/domains/{domainName}/relationship-types:
get:
  description: Get all the available Topology Relationship types.
  tags:
    - Entities and relationships
  summary: Get all the available topology relationship types.
  operationId: getTopologyRelationshipTypes
  parameters:
    - $ref: '#/components/parameters/acceptJsonInHeader'
    - $ref: '#/components/parameters/domainNameInPath'
    - $ref: '#/components/parameters/offsetParam'
    - $ref: '#/components/parameters/limitParam'
  responses:
    '200':
      description: OK
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/RelationshipTypes'
    '400':
      $ref: '#/components/responses/400'
    '401':
      $ref: '#/components/responses/401'
    '403':
      $ref: '#/components/responses/403'
    '404':
      $ref: '#/components/responses/404'

```

```
'406':
  $ref: '#/components/responses/406'
'414':
  $ref: '#/components/responses/414'
'429':
  $ref: '#/components/responses/429'
'500':
  $ref: '#/components/responses/500'
'502':
  $ref: '#/components/responses/502'
'503':
  $ref: '#/components/responses/503'
```

```
/domains/{domainName}/relationship-types/{relationshipTypeName}/relationships:
```

```
get:
```

```
description: Get all the available Topology Relationships of a specific relationship type name
```

```
tags:
```

```
- Entities and relationships
```

```
summary: Get all the available Topology Relationships of a specific relationship type name.
```

```
operationId: getRelationshipsByType
```

```
parameters:
```

```
- $ref: '#/components/parameters/acceptYangJsonInHeader'
- $ref: '#/components/parameters/domainNameInPath'
- $ref: '#/components/parameters/relationshipTypeNameInPath'
- $ref: '#/components/parameters/targetFilterOptionalInQuery'
- $ref: '#/components/parameters/scopeFilterOptionalInQuery'
- $ref: '#/components/parameters/offsetParam'
- $ref: '#/components/parameters/limitParam'
```

```
responses:
```

```
'200':
  description: OK
  content:
    application/json:
      schema:
        $ref: '#/components/schemas/Relationships'
'400':
  $ref: '#/components/responses/400'
'401':
  $ref: '#/components/responses/401'
'403':
  $ref: '#/components/responses/403'
'404':
  $ref: '#/components/responses/404'
'406':
  $ref: '#/components/responses/406'
'414':
  $ref: '#/components/responses/414'
'429':
  $ref: '#/components/responses/429'
'500':
  $ref: '#/components/responses/500'
'502':
  $ref: '#/components/responses/502'
'503':
  $ref: '#/components/responses/503'
```

```
/domains/{domainName}/relationship-types/{relationshipTypeName}/relationships/{relationshipId}:
```

```
get:
```

```
description: Get a specific Topology Relationship instance of a Topology Relationship type.
```

```
tags:
```

```
- Entities and relationships
```

```
summary: Get a specific Topology Relationship instance of a Topology Relationship type.
```

```
operationId: getRelationshipById
```

```
parameters:
```

```
- $ref: '#/components/parameters/acceptYangJsonInHeader'
- $ref: '#/components/parameters/domainNameInPath'
- $ref: '#/components/parameters/relationshipTypeNameInPath'
- $ref: '#/components/parameters/relationshipIdInPath'
```

```
responses:
```

```
'200':
  description: OK
```



```

        content:
          application/yang.data+json:
            schema:
              type: object
              description: Encapsulated object reference to the data model for schema definition of
Topology Relationship
      '400':
        $ref: '#/components/responses/400'
      '401':
        $ref: '#/components/responses/401'
      '403':
        $ref: '#/components/responses/403'
      '404':
        $ref: '#/components/responses/404'
      '406':
        $ref: '#/components/responses/406'
      '414':
        $ref: '#/components/responses/414'
      '429':
        $ref: '#/components/responses/429'
      '500':
        $ref: '#/components/responses/500'
      '502':
        $ref: '#/components/responses/502'
      '503':
        $ref: '#/components/responses/503'
/schemas:
  get:
    description: Get a list of all topology model schemas.
    tags:
      - Schemas
    summary: Get a list of all user defined schemas.
    operationId: getSchemas
    parameters:
      - $ref: '#/components/parameters/acceptJsonInHeader'
      - $ref: '#/components/parameters/domainOptionalInQuery'
      - $ref: '#/components/parameters/offsetParam'
      - $ref: '#/components/parameters/limitParam'
    responses:
      '200':
        description: OK
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/Schemas'
      '400':
        $ref: '#/components/responses/400'
      '401':
        $ref: '#/components/responses/401'
      '403':
        $ref: '#/components/responses/403'
      '404':
        $ref: '#/components/responses/404'
      '406':
        $ref: '#/components/responses/406'
      '414':
        $ref: '#/components/responses/414'
      '429':
        $ref: '#/components/responses/429'
      '500':
        $ref: '#/components/responses/500'
      '502':
        $ref: '#/components/responses/502'
      '503':
        $ref: '#/components/responses/503'
/schemas/{schemaName}/content:
  get:
    description: Get the model schema by name.
    tags:
      - Schemas
    summary: Get the model schema.

```

```

operationId: getSchemaByName
parameters:
  - $ref: '#/components/parameters/acceptPlainTextInHeader'
  - $ref: '#/components/parameters/schemaNameInPath'
responses:
  '200':
    description: OK
    content:
      text/plain:
        schema:
          type: string
  '400':
    $ref: '#/components/responses/400'
  '401':
    $ref: '#/components/responses/401'
  '403':
    $ref: '#/components/responses/403'
  '404':
    $ref: '#/components/responses/404'
  '406':
    $ref: '#/components/responses/406'
  '414':
    $ref: '#/components/responses/414'
  '429':
    $ref: '#/components/responses/429'
  '500':
    $ref: '#/components/responses/500'
  '502':
    $ref: '#/components/responses/502'
  '503':
    $ref: '#/components/responses/503'
/user-defined-schemas:
  post:
    description: Create a new user defined schema. The request body contains the schema in YANG format.
    tags:
      - User Defined Schemas
    summary: Create a new user defined schema.
    operationId: createUserDefinedSchema
    parameters:
      - $ref: '#/components/parameters/acceptJsonInHeader'
      - $ref: '#/components/parameters/contentTypeMultipartFileInHeader'
    requestBody:
      required: true
      content:
        multipart/form-data:
          schema:
            $ref: '#/components/schemas/MultipartFile'
    responses:
      '201':
        description: Created
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/UserDefinedSchema'
        headers:
          Location:
            description: Contains the URI of the newly created resource
            required: true
            schema:
              type: string
      '400':
        $ref: '#/components/responses/400'
      '401':
        $ref: '#/components/responses/401'
      '403':
        $ref: '#/components/responses/403'
      '404':
        $ref: '#/components/responses/404'
      '409':
        $ref: '#/components/responses/409'
      '411':

```

```

    $ref: '#/components/responses/411'
  '413':
    $ref: '#/components/responses/413'
  '415':
    $ref: '#/components/responses/415'
  '429':
    $ref: '#/components/responses/429'
  '500':
    $ref: '#/components/responses/500'
  '502':
    $ref: '#/components/responses/502'
  '503':
    $ref: '#/components/responses/503'
get:
  description: Get a list of all user defined schemas.
  tags:
    - User Defined Schemas
  summary: Get a list of all user defined schemas.
  operationId: getUserDefinedSchemas
  parameters:
    - $ref: '#/components/parameters/acceptJsonInHeader'
    - $ref: '#/components/parameters/offsetParam'
    - $ref: '#/components/parameters/limitParam'
  responses:
    '200':
      description: OK
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/UserDefinedSchemas'
    '400':
      $ref: '#/components/responses/400'
    '401':
      $ref: '#/components/responses/401'
    '403':
      $ref: '#/components/responses/403'
    '404':
      $ref: '#/components/responses/404'
    '406':
      $ref: '#/components/responses/406'
    '414':
      $ref: '#/components/responses/414'
    '429':
      $ref: '#/components/responses/429'
    '500':
      $ref: '#/components/responses/500'
    '502':
      $ref: '#/components/responses/502'
    '503':
      $ref: '#/components/responses/503'
/user-defined-schemas/{schemaName}/content:
get:
  description: Get the user defined schema by name.
  tags:
    - User Defined Schemas
  summary: Get the user defined schema.
  operationId: getUserDefinedSchemaByName
  parameters:
    - $ref: '#/components/parameters/acceptPlainTextInHeader'
    - $ref: '#/components/parameters/schemaNameInPath'
  responses:
    '200':
      description: OK
      content:
        text/plain:
          schema:
            type: string
    '400':
      $ref: '#/components/responses/400'
    '401':
      $ref: '#/components/responses/401'

```

```

'403':
  $ref: '#/components/responses/403'
'404':
  $ref: '#/components/responses/404'
'406':
  $ref: '#/components/responses/406'
'414':
  $ref: '#/components/responses/414'
'429':
  $ref: '#/components/responses/429'
'500':
  $ref: '#/components/responses/500'
'502':
  $ref: '#/components/responses/502'
'503':
  $ref: '#/components/responses/503'
/user-defined-schemas/{schemaName}:
  delete:
    description: Delete a user defined schema.
    tags:
      - User Defined Schemas
    summary: Delete a user defined schema.
    operationId: deleteUserDefinedSchema
    parameters:
      - $ref: '#/components/parameters/acceptJsonInHeader'
      - $ref: '#/components/parameters/schemaNameInPath'
    responses:
      '204':
        $ref: '#/components/responses/204'
      '400':
        $ref: '#/components/responses/400'
      '401':
        $ref: '#/components/responses/401'
      '403':
        $ref: '#/components/responses/403'
      '404':
        $ref: '#/components/responses/404'
      '429':
        $ref: '#/components/responses/429'
      '500':
        $ref: '#/components/responses/500'
      '502':
        $ref: '#/components/responses/502'
      '503':
        $ref: '#/components/responses/503'

/manage-classifiers:
  post:
    description: Update entities and/or relationships with classifier(s).
    tags:
      - Classifiers
    summary: Update entities and/or relationships with classifier(s).
    operationId: updateClassifier
    parameters:
      - $ref: '#/components/parameters/acceptJsonInHeader'
      - $ref: '#/components/parameters/contentTypeJsonInHeader'
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/Classifier'
    responses:
      '204':
        $ref: '#/components/responses/204'
      '400':
        $ref: '#/components/responses/400'
      '401':
        $ref: '#/components/responses/401'
      '403':
        $ref: '#/components/responses/403'

```

```

    '404':
      $ref: '#/components/responses/404'

    '411':
      $ref: '#/components/responses/411'
    '413':
      $ref: '#/components/responses/413'
    '415':
      $ref: '#/components/responses/415'
    '429':
      $ref: '#/components/responses/429'
    '500':
      $ref: '#/components/responses/500'
    '502':
      $ref: '#/components/responses/502'
    '503':
      $ref: '#/components/responses/503'

/manage-decorators:
  post:
    description: Update entities and/or relationships with decorator(s).
    tags:
      - Decorators
    summary: Update entities and/or relationships with decorator(s).
    operationId: updateDecorator
    parameters:
      - $ref: '#/components/parameters/acceptJsonInHeader'
      - $ref: '#/components/parameters/contentTypeJsonInHeader'
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/Decorator'
    responses:
      '204':
        $ref: '#/components/responses/204'
      '400':
        $ref: '#/components/responses/400'
      '401':
        $ref: '#/components/responses/401'
      '403':
        $ref: '#/components/responses/403'
      '404':
        $ref: '#/components/responses/404'

      '411':
        $ref: '#/components/responses/411'
      '413':
        $ref: '#/components/responses/413'
      '415':
        $ref: '#/components/responses/415'
      '429':
        $ref: '#/components/responses/429'
      '500':
        $ref: '#/components/responses/500'
      '502':
        $ref: '#/components/responses/502'
      '503':
        $ref: '#/components/responses/503'

components:
  schemas:
    Classifier:
      type: object
      title: Classifier
      properties:
        operation:
          type: string
          enum:
            - merge

```

```

    - delete
  classifiers:
    type: array
    items:
      type: string
  entityIds:
    type: array
    items:
      type: string
  relationshipIds:
    type: array
    items:
      type: string
Decorator:
  type: object
  title: Decorator
  properties:
    operation:
      type: string
      enum:
        - merge
        - delete
    decorators:
      type: object
      additionalProperties: true
      description: Decorators must be defined in schema before use. Data type of a decorator is
restricted as defined by its schema.
    entityIds:
      type: array
      items:
        type: string
    relationshipIds:
      type: array
      items:
        type: string
Domains:
  type: object
  title: Domains
  properties:
    items:
      type: array
      items:
        type: object
        properties:
          domainName:
            type: string
          entityTypes:
            $ref: '#/components/schemas/Href'
          relationshipTypes:
            $ref: '#/components/schemas/Href'
          required: ['domainName', 'entityTypes', 'relationshipTypes']
        required: ['items']
EntityTypes:
  type: object
  title: EntityTypes
  properties:
    items:
      type: array
      items:
        type: object
        properties:
          entityType:
            type: string
          entities:
            $ref: '#/components/schemas/Href'
          required: ['entityType', 'entities']
        required: ['items']
Entities:
  type: object
  title: Entities
  properties:

```

```

    items:
      type: array
    items:
      type: object
      description: Encapsulated object reference to the data model for schema definition of Topology
Entity
  required: ['items']
RelationshipTypes:
  type: object
  title: RelationshipTypes
  properties:
    items:
      type: array
    items:
      type: object
      properties:
        relationshipTypeName:
          type: string
        relationships:
          $ref: '#/components/schemas/Href'
          required: ['relationshipTypeName', 'relationships']
      required: ['items']
Relationships:
  type: object
  title: Relationships
  properties:
    items:
      type: array
    items:
      type: object
      description: Encapsulated object reference to the data model for schema definition of Topology
Relationships
  required: ['items']
ProblemDetails:
  description: A problem detail to carry details in an HTTP response according to RFC 7807
  type: object
  properties:
    type:
      description: a URI reference according to IETF RFC 3986 that identifies the problem type
      type: string
    title:
      description: human-readable summary of the problem type
      type: string
    status:
      description: the HTTP status code
      type: number
    detail:
      description: human-readable explanation
      type: string
    instance:
      description: URI reference that identifies the specific occurrence of the problem
      type: string
Href:
  type: object
  title: Href
  properties:
    href:
      type: string
      format: uri-template
MultipartFile:
  type: object
  required:
    - file
  properties:
    file:
      type: string
      description: multipartFile
      format: binary
Schema:
  type: object
  title: Schema

```

```

    properties:
      name:
        type: string
      domain:
        type: string
      revision:
        type: string
      content:
        $ref: '#/components/schemas/Href'
  Schemas:
    type: object
    title: Schemas
    properties:
      items:
        type: array
        items:
          $ref: '#/components/schemas/Schema'
  UserDefinedSchema:
    type: object
    title: UserDefinedSchema
    properties:
      name:
        type: string
      domain:
        type: string
      revision:
        type: string
      content:
        $ref: '#/components/schemas/Href'
  UserDefinedSchemas:
    type: object
    title: UserDefinedSchemas
    properties:
      items:
        type: array
        items:
          $ref: '#/components/schemas/UserDefinedSchema'
  responses:
    '204':
      description: No Content
      content: {}
    '400':
      description: Bad Request
      content:
        application/problem+json:
          schema:
            $ref: '#/components/schemas/ProblemDetails'
    '401':
      description: Unauthorized
      content:
        application/problem+json:
          schema:
            $ref: '#/components/schemas/ProblemDetails'
    '403':
      description: Forbidden
      content:
        application/problem+json:
          schema:
            $ref: '#/components/schemas/ProblemDetails'
    '404':
      description: Not Found
      content:
        application/problem+json:
          schema:
            $ref: '#/components/schemas/ProblemDetails'
    '406':
      description: Not Acceptable
      content:
        application/problem+json:
          schema:

```



```

    $ref: '#/components/schemas/ProblemDetails'
'409':
  description: Conflict
  content:
    application/problem+json:
      schema:
        $ref: '#/components/schemas/ProblemDetails'
'411':
  description: Length Required
  content:
    application/problem+json:
      schema:
        $ref: '#/components/schemas/ProblemDetails'
'413':
  description: Payload Too Large
  content:
    application/problem+json:
      schema:
        $ref: '#/components/schemas/ProblemDetails'
'414':
  description: URI Too Large
  content:
    application/problem+json:
      schema:
        $ref: '#/components/schemas/ProblemDetails'
'415':
  description: Unsupported Media Type
  content:
    application/problem+json:
      schema:
        $ref: '#/components/schemas/ProblemDetails'
'429':
  description: Too Many Requests
  content:
    application/problem+json:
      schema:
        $ref: '#/components/schemas/ProblemDetails'
'500':
  description: Internal Server Error
  content:
    application/problem+json:
      schema:
        $ref: '#/components/schemas/ProblemDetails'
'502':
  description: Bad Gateway
  content:
    application/problem+json:
      schema:
        $ref: '#/components/schemas/ProblemDetails'
'503':
  description: Service Unavailable
  content:
    application/problem+json:
      schema:
        $ref: '#/components/schemas/ProblemDetails'
parameters:
  acceptPlainTextInHeader:
    name: Accept
    in: header
    required: true
    schema:
      type: string
      default: text/plain
  contentTypeMultipartFileInHeader:
    name: Content-Type
    in: header
    required: true
    schema:
      type: string
      default: multipart/form-data
  domainOptionalInQuery:

```

```

    name: domain
    in: query
    required: false
    schema:
      type: string
  schemaNameInPath:
    name: schemaName
    in: path
    required: true
    schema:
      type: string
  contentTypeJsonInHeader:
    name: Content-Type
    in: header
    required: true
    schema:
      type: string
      default: application/json
  acceptJsonInHeader:
    name: Accept
    in: header
    required: true
    schema:
      type: string
      default: application/json
  acceptYangJsonInHeader:
    name: Accept
    in: header
    required: true
    schema:
      type: string
      default: application/yang.data+json
  offsetParam:
    name: offset
    in: query
    description: This query parameter allows you to omit a specified number of entries before the
beginning of the result set for pagination.
    required: false
    schema:
      type: integer
      default: 0
      minimum: 0
  limitParam:
    name: limit
    in: query
    description: The query parameter provides to limit the number of entries returned for a request for
pagination.
    required: false
    schema:
      type: integer
      default: 500
      minimum: 1
      maximum: 500
  domainNameInPath:
    name: domainName
    in: path
    description: domain name
    required: true
    schema:
      type: string
  entityIdInPath:
    name: entityId
    in: path
    required: true
    schema:
      type: string
  relationshipIdInPath:
    name: relationshipId
    in: path
    required: true
    schema:

```

```
    type: string
entityTypeNameInPath:
  name: entityTypeNames
  in: path
  required: true
  schema:
    type: string
relationshipTypeNameInPath:
  name: relationshipTypeNames
  in: path
  required: true
  schema:
    type: string
targetFilterOptionalInQuery:
  name: targetFilter
  description: Use *targetFilter* to specify the entity type and
    attributes to be returned in the REST response. The value for
    *targetFilter* can also be a list of entity types and attributes.
  in: query
  required: false
  schema:
    type: string
scopeFilterOptionalInQuery:
  name: scopeFilter
  description: Use *scopeFilter* to specify the attributes to match on.
    The value for *scopeFilter* can also be a list of entity types and
    attributes. scopeFilter returns a boolean.
  in: query
  required: false
  schema:
    type: string
```

Annex (informative): Change History

Date	Revision	Description
2024.10.01	00.00.01	Initial proposed skeleton for the TE&IV Application Protocols Specification
2024.11.21	01.00	Implemented the following CRs for the November 2024 train: ERI-2024.08.14-WG10-CR-0110-TE&IV_API_Content-v02, ERI-2024.09.23-WG10-CR-0115-TE&IV_API_Service operation and Resources-v02, ERI-2024.10.16-WG10-CR-0125-TE&IVOpenAPI-v01, ERI-2024.11.07-WG10-CR-0126-TE&IV_API_version_update-v01
2025.02.07	02.00	Implemented the following CRs for the March 2025 train: ERI-2024.12.10-WG10-CR-0141-Adding Yang+Json to Content-v01, ERI-2025.02.17-WG10-CR-0149-stage 3 to add query parameter to topology entity relationship resource -v02, ERI-2025.02.17-WG10-CR-0151-OpenAPI update to add query parameter to topology entity relationship resource -v01
2025.07.01	03.00	Implemented the following CRs for the July 2025 train: ERI-2025.05.20-WG10-CR-0175-Operations-for-Classifiers-and-Decorators-v06, ERI-2025.06.23-WG10-CR-0178-Schema-Operations-v01. Updated the specification for formatting corrections and editorial updates based on ODR and TS template v04
2025.11.19	04.00	Implemented the following CRs for the November 2025 train: ERI-2025.07.21-WG10-CR-0191-Removing Unused Error Codes-v01, ERI-2025.11.06-WG10-CR-0212-Replacing O-RAN Alliance license with SCCL license in TE&IV OpenAPI-v01, Editorial fixes, Incremented the patch version of the TE&IV API to implement the CR 191